time when a block can be located and read on the average within 60 ms. and the time to search each record is one ms. per block? Compare this time to the time required to search a single block for the same information.

11.6. Referring to Problem 11.4, describe how a hashing method could be applied to search for the indicated records.

11.7. Draw a conceptual indexing tree structure using the same keys as those given in Problem 11.4, but with the addition of a generalized node named farm-animals.

11.8. Using the same label links as those used in HAM, develop propositional trees for the following sentences.
The birds were singing in the park.
John and Mary went dancing at the prom.
Do not drink the water.

11.9. For the previous problem, add the sentence "There are lots of birds and they are small and yellow."

11.10. Develop an E-MOP for a general episode to fill up a car with gasoline using the elements Actor, Participant, Objects, Actions, and Goals.

11.11. Show how the E-MOP of Problem 11.10 would be indexed and accessed for the two events of filling the car at a self-service and at a full-service location.

11.12. Are the events of Problem 11.11 good candidates for specialized E-MOPs? Explain your answer.

11.13. Give an example of a hashing function that does not distribute key values uniformly over the key space.

11.14. Draw a small hypertext network that you might want to browse where the general network subject of artificial intelligence is used. Make up your own subtopics and show all linkages which you feel are useful, including link directions between subtopics.

11.15. Show how the E-MOP of Figure 11.7 would be generalized when peace was one of the topics discussed at every meeting.

11.16. Modify the E-MOP of Figure 11.7 to accommodate a new meeting between Vance and King Hussain of Jordan. The topic of their meeting is Palestinian refugees.

PART 4
Perception, Communication, and Expert Systems

12
Natural Language Processing

Perception and communication are essential components of intelligent behavior. They provide the ability to effectively interact with our environment. Humans perceive and communicate through their five basic senses of sight, hearing, touch, smell, and taste, and their ability to generate meaningful utterances. Two of the senses, sight and hearing are especially complex and require concious inferencing. Developing programs that understand natural language and that comprehend visual scenes are two of the most difficult tasks facing AI researchers.

Developing programs that understand a natural language is a difficult problem. Natural languages are large. They contain an infinity of different sentences. No matter how many sentences a person has heard or seen, new ones can always be produced. Also, there is much ambiguity in a natural language. Many words have several meanings such as can, bear, fly, and orange, and sentences can have different meanings in different contexts. This makes the creation of programs that "understand" a natural language, one of the most challenging tasks in AI. It requires that a program transform sentences occurring as part of a dialog into data structures which convey the intended meaning of the sentences to a reasoning program. In general, this means that the reasoning program must know a lot about the structure of the language, the possible semantics, the beliefs and goals of the user, and a great deal of general world knowledge.

12.1 INTRODUCTION

Developing programs to understand natural language is important in AI because a natural form of communication with systems is essential for user acceptance. Furthermore, one of the most critical tests for intelligent behavior is the ability to communicate effectively. Indeed, this was the purpose of the test proposed by Alan Turing (see Chapter 2). AI programs must be able to communicate with their human counterparts in a natural way, and natural language is one of the most important mediums for that purpose.

Before proceeding further, a definition of understanding as used here should be given. We say a program *understands* a natural language if it behaves by taking a (predictably) correct or acceptable action in response to the input. For example, we say a child demonstrates understanding if it responds with the correct answer to a question. The action taken need not be an external response. It may simply be the creation of some internal data structures as would occur in learning some new facts. But in any case, the structures created should be meaningful and correctly interact with the world model representation held by the program. In this chapter we explore many of the important issues related to natural language understanding and language generation.

12.2 OVERVIEW OF LINGUISTICS

An understanding of linguistics is not a prerequisite to the study of natural language understanding, but a familiarity with the basics of grammar is certainly important. We must understand how words and sentences are combined to produce meaningful word strings before we can expect to design successful language understanding systems. In a natural language, the sentence is the basic language element. A sentence is made up of words which express a complete thought. To express a complete thought, a sentence must have a subject and a predicate. The subject is what the sentence is about, and the predicate says something about the subject.

Sentences are classified by structure and usage. A simple sentence has one independent clause comprised of a subject and predicate. A compound sentence consists of two or more independent clauses connected by a conjunction or a semicolon. A complex sentence consists of an independent clause and one or more dependent clauses. Sentences are used to assert, query, and describe. The way a sentence is used determines its mood, declarative, imperative, interrogative, or exclamatory.

A word functions in a sentence as a part of speech. Parts of speech for the English language are nouns, pronouns, verbs, adjectives, adverbs, prepositions, conjunctions, and interjections.

A noun is a name for something (person, place, or thing). Pronouns replace nouns when the noun is already known. Verbs express action, being, or state of being. Adjectives are used to modify nouns and pronouns, and adverbs modify verbs, adjectives, or other adverbs. Prepositions establish the relationship between

a noun and some other part of the sentence. Conjunctions join words or groups of words together, and interjections are used to express strong feelings apart from the rest of the sentence.

Phrases are made up of words but act as a single unit within a sentence. These form the building blocks for the syntactic structures we consider later.

Levels of Knowledge Used in Language Understanding

A language understanding program must have considerable knowledge about the structure of the language including what the words are and how they combine into phrases and sentences. It must also know the meanings of the words and how they contribute to the meanings of a sentence and to the context within which they are being used. Finally, a program must have some general world knowledge as well as knowledge of what humans know and how they reason. To carry on a conversation with someone requires that a person (or program) know about the world in general, know what other people know, and know the facts pertaining to a particular conversational setting. This all presumes a familiarity with the language structure and a minimal vocabulary.

The component forms of knowledge needed for an understanding of natural language are sometimes classified according to the following levels.

Phonological. This is knowledge which relates sounds to the words we recognize. A phoneme is the smallest unit of sound. Phones are aggregated into word sounds.

Morphological. This is lexical knowledge which relates to word constructions from basic units called morphemes. A morpheme is the smallest unit of meaning; for example, the construction of friendly from the root *friend* and the suffix *ly*.

Syntactic. This knowledge relates to how words are put together or structured to form grammatically correct sentences in the language.

Semantic. This knowledge is concerned with the meanings of words and phrases and how they combine to form sentence meanings.

Pragmatic. This is high-level knowledge which relates to the use of sentences in different contexts and how the context affects the meaning of the sentences.

World. World knowledge relates to the language a user must have in order to understand and carry on a conversation. It must include an understanding of the other person's beliefs and goals.

The approaches taken in developing language understanding programs generally follow the above levels or stages. When a string of words has been detected, the

sentences are parsed or analyzed to determine their structure (syntax) and grammatical correctness. The meanings (semantics) of the sentences are then determined and appropriate representation structures created for the inferencing programs. The whole process is a series of transformations from the basic speech sounds to a complete set of internal representation structures.

Understanding written language or text is easier than understanding speech. To understand speech, a program must have all the capabilities of a text understanding program plus the facilities needed to map spoken sounds (often corrupted by noise) into textual form. In this chapter, we focus on the easier problem, that of natural language understanding from textual input and information processing. The process of translating speech into written text is considered in Chapter 13 under Pattern Recognition and the process of generating text is considered later in this chapter.

General Approaches to Natural Language Understanding

Essentially, there have been three different approaches taken in the development of natural language understanding programs, (1) the use of keyword and pattern matching, (2) combined syntactic (structural) and semantic directed analysis, and (3) comparing and matching the input to real world situations (scenario representations).

The keyword and pattern matching approach is the simplest. This approach was first used in programs such as ELIZA described in Chapter 10. It is based on the use of sentence templates which contain key words or phrases such as "_____ my mother _____," "I am _____," and, "I don't like _____," that are matched against input sentences. Each input template has associated with it one or more output templates, one of which is used to produce a response to the given input. Appropriate word substitutions are also made from the input to the output to produce the correct person and tense in the response (I and me into you to give replies like "Why are you _____"). The advantage of this approach is that ungrammatical, but meaningful sentences are still accepted. The disadvantage is that no actual knowledge structures are created; so the program does not really understand.

The third approach is based on the use of structures such as the frames or scripts described in Chapter 7. This approach relies more on a mapping of the input to prescribed primitives which are used to build larger knowledge structures. It depends on the use of constraints imposed by context and world knowledge to develop an understanding of the language inputs. Prestored descriptions and details for commonly occurring situations or events are recalled for use in understanding a new situation. The stored events are then used to fill in missing details about the current scenario. We will be returning to this approach later in this chapter. Its advantage is that much of the computation required for syntactical analysis is bypassed. The disadvantage is that a substantial amount of specific, as well as general world knowledge must be prestored.

The second approach is one of the most popular approaches currently being

used and is the main topic of the first part of this chapter. With this approach, knowledge structures are constructed during a syntactical and semantical analysis of the input sentences. Parsers are used to analyze individual sentences and to build structures that can be used directly or transformed into the required knowledge formats. The advantage of this approach is in the power and versatility it provides. The disadvantage is the large amount of computation required and the need for still further processing to understand the contextual meanings of more than one sentence.

12.3 GRAMMARS AND LANGUAGES

A language L can be considered as a set of strings of finite or infinite length, where a string is constructed by concatenating basic atomic elements called symbols. The finite set v of symbols of the language is called the alphabet or vocabulary. Among all possible strings that can be generated from v are those that are well-formed, the sentences (such as the sentences found in a language like English). Well-formed sentences are constructed using a set of rules called a grammar. A grammar G is a formal specification of the sentence structures that are allowable in the language, and the language generated by the grammar G is denoted by L(G).

More formally, we define a grammar G as

$$G = (v_n, v_t, s, p)$$

where v_n is a set of nonterminal symbols, v_t a set of terminal symbols, s is a starting symbol, and p is a finite set of productions or rewrite rules. The alphabet v is the union of the disjoint sets v_n and v_t which includes the empty string e. The terminals v_t, are symbols which cannot be decomposed further (such as adjectives, nouns or verbs in English), whereas the nonterminals can be decomposed (such as a noun or verb phrase).

A general production rule from P has the form

$$xyz \rightarrow xwz$$

where x, y, z, and w are strings from v. This rule states that y should be rewritten as w in the context of x to z where x and z can be any string including the empty string e.

As an example of a simple grammar G, we choose one which has component parts or constituents from English with vocabulary Q given by

$$Q_N = \{S, NP, N, VP, V, ART\}$$
$$Q_T = \{boy, popsicle, frog, ate, kissed, flew, the, a\}$$

and rewrite rules given by

$$
\begin{aligned}
P: \quad & S \rightarrow NP\ VP \\
& NP \rightarrow ART\ N \\
& VP \rightarrow V\ NP \\
& N \rightarrow \text{boy} \mid \text{popsicle} \mid \text{frog} \\
& V \rightarrow \text{ate} \mid \text{kissed} \mid \text{flew} \\
& ART \rightarrow \text{the} \mid \text{a}
\end{aligned}
$$

where the vertical bar indicates alternative choices.

S is the initial symbol (for sentence here), NP stands for noun phrase, VP stands for verb phrase, N stands for noun, V is an abbreviation for verb, and ART stands for article.

The grammar G defined above generates only a small fraction of English, but it illustrates the general concepts of generative grammars. With this G, sentences such as the following can be generated.

The boy ate a popsicle.

The frog kissed a boy.

A boy ate the frog.

To generate a sentence, the rules from P are applied sequentially starting with S and proceeding until all nonterminal symbols are eliminated. As an example, the first sentence given above can be generated using the following sequence of rewrite rules:

$$
\begin{aligned}
S &\rightarrow NP\ VP \\
&\rightarrow ART\ N\ VP \\
&\rightarrow \text{the}\ N\ VP \\
&\rightarrow \text{the boy}\ VP \\
&\rightarrow \text{the boy}\ V\ NP \\
&\rightarrow \text{the boy ate}\ NP \\
&\rightarrow \text{the boy ate}\ ART\ N \\
&\rightarrow \text{the boy ate a}\ N \\
&\rightarrow \text{the boy ate a popsicle}
\end{aligned}
$$

It should be clear that a grammar does not guarantee the generation of meaningful sentences, only that they are structurally correct. For example, a gramatically correct, but meaningless sentence like "The popsicle flew a frog" can be generated with this grammar.

We learn a language by learning its structure and not by memorizing all of the sentences we have ever heard, and we are able to use the language in a variety of ways because of this familiarity. Therefore, a useful model of language is one which characterizes the permissible structures through the generating grammars. Unfortunately, it has not been possible to formally characterize natural languages with a simple grammar. In other words, it has not been possible to classify natural languages in a mathematical sense as we did in the example above. More constrained

languages (formal programming languages) have been classified and studied through the use of similar grammars, including the Chomsky classes of languages (1965).

The Chomsky Hierarchy of Generative Grammars

Noam Chomsky defined a hierarchy of grammars he called types 0, 1, 2, and 3. Type 0 grammar is the most general. It is obtained by making the simple restriction that y cannot be the empty string in the rewrite form $xyz \rightarrow xwz$. This broad generality requires that a computer having the power of a Turing machine be used to recognize sentences of type 0.

The next level down in generality is obtained with type 1 grammars which are called context-sensitive grammars. They have the added restriction that the length of the string on the right-hand side of the rewrite rule must be at least as long as the string on the left-hand side. Furthermore, in productions of the form $xyz \rightarrow xwz$, y must be a single nonterminal symbol and w, a nonempty string. Typical rewrite rules for type 1 grammars take the forms

$$S \rightarrow aS$$
$$S \rightarrow aAB$$
$$AB \rightarrow BA$$
$$aA \rightarrow ab$$
$$aA \rightarrow aa$$

where the capitalized letters are nonterminals and the lower case letters terminals.

The third type, the type 2 grammar, is known as a context-free grammar. It is characterized by rules with the general form <symbol> → <symbol1> . . . <symbolk> where $k \geq 1$ and where the left-hand side is a single nonterminal symbol, $A \rightarrow xyz$ where A is a single nonterminal. Productions for this type take forms such as

$$S \rightarrow aS$$
$$S \rightarrow aSb$$
$$S \rightarrow aB$$
$$S \rightarrow aAB$$
$$A \rightarrow a$$
$$B \rightarrow b$$

The final and most restrictive type is type 3. It is also called a finite state or regular grammar, whose rules are characterized by the forms

$$A \rightarrow aB$$
$$A \rightarrow a$$

The languages generated by these grammars are also termed types 0, 1 (context-sensitive), 2 (context-free), and 4 (regular) corresponding to the grammars which generate them.

The regular and context-free languages have been the most widely studied

and best understood. For example, formal programming languages are typically based on context-free languages. Consequently, much of the work in human language understanding has been related to these two types. This is understandable since type 0 grammars are too general to be of much practical use, and type 1 grammars are not that well understood yet.

Structural Representations

It is convenient to represent sentences as a tree or graph to help expose the structure of the constituent parts. For example, the sentence "The boy ate a popsicle" can be represented as the tree structure depicted in Figure 12.1. Such structures are also called phrase markers because they help to mark or identify the phrase structures in a sentence.

The root node of the tree in Figure 12.1 corresponds to the whole sentence S, and the constituent parts of S are subtrees. For this example, the left subtree is a noun phrase, and the right subtree a verb phrase. The leaf or terminal nodes contain the terminal symbols from V_t.

A tree structure such as the above represents a large number of English sentences. It also represents a large class of ill-formed strings that are nonsentences like "The popsicle flew a tree." This satisfies the above structure, but has no meaning.

For purposes of computation, a tree must be represented as a record, a list or similar data structure. We saw in earlier chapters that a list can always be used to represent a tree structure. For example, the tree in Figure 12.1 could be represented as the list

```
(S  (NP  ((ART the)
          (N boy))
     (VP  (V ate)
          (NP  (ART a)
               (N popsicle)))))
```

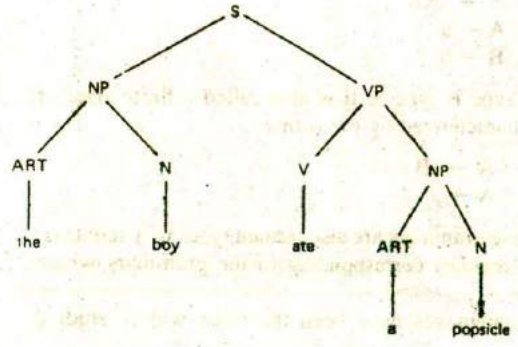


Figure 12.1 A phrase marker or syntactic tree.

A more extensive English grammar than the one given above can be obtained with the addition of other constituents such as prepositional phrases PP, adjectives ADJ, determiners DET, adverbs ADV, auxiliary verbs AUX, and so on. Additional rewrite rules permitting the use of these constituents could include some of the following:

$$\begin{aligned} PP &\rightarrow PREP\ NP \\ VP &\rightarrow V\ ADV \\ VP &\rightarrow V\ PP \\ VP &\rightarrow V\ NP\ PP \\ VP &\rightarrow AUX\ V\ NP \\ DET &\rightarrow ART\ ADJ \\ DET &\rightarrow ART \end{aligned}$$

These extensions broaden the types of sentences that can be generated by permitting the added constituents in sentence forms such as

The mean boy locked the dog in the house.
The cute girl worked to make some extra money.

These sentences have the form $S \rightarrow NP\ VP\ PP$.

Transformational Grammars

The generative grammars described above generally produce different structures for sentences having different syntactical forms even though they may have the same semantic content. For example, the active and passive forms of a sentence will result in two different phrase marker structures. The sentences "Joe kissed Sue" (active voice) and "Sue was kissed by Joe" (passive voice) result in the structures depicted in Figure 12.2 where the subject and object roles for Joe and Sue are switched.

Obtaining different structures from sentences having the same meaning is undesirable in language understanding systems. Sentences with the same meaning should always map to the same internal knowledge structures. In an attempt to repair these shortcomings in generative grammars, Chomsky (1965) extended them by incorporating two additional components to the basic syntactic component. The added components provide a mechanism to produce single representations for sentences having the same meanings through a series of transformations. This extended grammar is called a transformational generative grammar. Its additions include a semantic component and a phonological component. They are used to interpret the output of the syntactic component by producing meanings and sound sequences. The transformations are essentially tree manipulation rules which depend on the use of an extended lexicon (dictionary) containing a number of semantic features for each word.

Using a transformational generative grammar, a sentence is analyzed in two stages. In one stage the basic structure of the sentence is analyzed to determine the

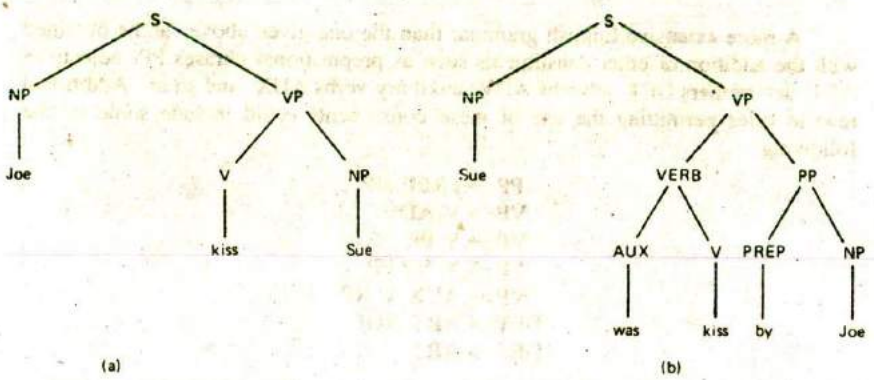


Figure 12.2 Structures for (a) active and (b) passive voice.

grammatical constituent parts. This reveals the surface structure of the sentence, the way the sentence is used in speech or in writing. This structure can be transformed into another one where the deeper semantic structure of the sentence is determined.

Application of the transformation rules can produce a change from passive voice to active voice, change a question to declarative form, and handle negations, subject-verb agreement, and so on. For example, the structure in 12.2(b) could be transformed to give the same basic structure as that of 12.2(a) as is illustrated in Figure 12.3.

Transformational grammars were never widely adopted as computational models of natural language. Instead, other grammars, including case grammars, have had more influence on such models.

Case Grammars

A case relates to the semantic role that a noun phrase plays with respect to verbs and adjectives. Case grammars use the functional relationships between noun phrases and verbs to reveal the deeper case of a sentence. These grammars use the fact

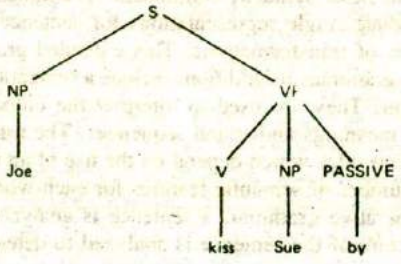


Figure 12.3 Passive voice transformed to active voice.

that verbal elements provide the main source of structure in a sentence since they describe the subject and objects.

In inflected languages like Latin, nouns generally have different ending forms for different cases. In English these distinctions are less pronounced and the forms remain more constant for different cases. Even so, they provide some constraints. English cases are the nominative (subject of the verb), possessive (showing possession or ownership), and objective (direct and indirect objects). Fillmore (1968, 1977) revived the notion of using case to extract the meanings of sentences. He extended the transformational grammars of Chomsky by focusing more on the semantic aspects of a sentence.

In case grammars, a sentence is defined as being composed of a proposition P, a tenseless set of relationships among verbs and noun phrases and a modality constituent M, composed of mood, tense, aspect, negation, and so on. Thus, a sentence can be represented as

$$S \rightarrow M + P$$

where P in turn consists of one or more distinct cases C1, C2, . . . , Ck,

$$P \rightarrow C1 + C2 + \ldots + Ck.$$

The number of cases suggested by Fillmore were relatively few. For example, the original list contained only some six cases. They relate to the actions performed by agents, the location and direction of actions, and so on. For example, the case of an instigator of an action is the agentive (or agent), the case of an instrument or object used in an action is the instrumental, and the case of the object receiving the action or change is the objective. Thus, in sentences like "The soldier struck the suspect with the rifle butt" the soldier is the agentive case, the suspect the objective case, and the rifle butt the instrumental case. Other basic cases include dative (an animate entity affected by an action), factitive (the case of the object or of being that which results from an event), and locative (the case of location of the event). Additional cases or substitutes for those given above have since been introduced, including beneficiary, source, destination, to or from, goal, and time.

Case frames are provided for verbs to identify allowable cases. They give the relationships which are required and those which are optional. For the above sentence, a case frame for the verb struck might be

STRUCK[OBJECTIVE (AGENTIVE) (INSTRUMENTAL)]

This may be interpreted as stating that the verb struck must occur in sentences with a noun phrase in the objective case and optionally (parentheses indicate optional use) with noun phrases in the agentive and instrumental cases.

A tree representation for a case grammar will identify the words by their modality and case. For example, a case grammar tree for the sentence "Sue did not take the car" is illustrated in Figure 12.4.

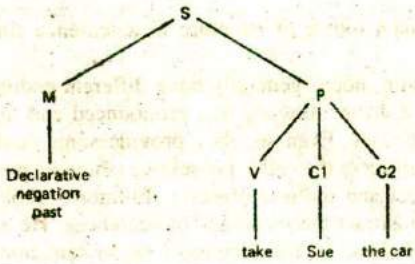


Figure 12.4 Case grammar tree representation.

To build a tree structure like this requires that a word lexicon with sufficient information be available in which to determine the case of sentence elements.

Systemic Grammars

Systemic grammars emphasize function and purpose in the analysis of language. They attempt to account for the personal and social aspects which influence communication through language. As such, context and pragmatics play a more important role in systemic grammars.

Systemic grammars were introduced by Michael Halliday (1961) and Winograd (1972) in an attempt to account for the principles that underlie the organization of different structures. He classifies language by three functions which relate to content, purpose, and coherence.

1. The *ideational function* relates to the content intended by the speaker. This function provides information about the kinds of activities being described, who the actors are, whether there are other participants, and the circumstances related to time and place. These concepts bear some similarity to the case grammars described above.

2. The *interpersonal function* is concerned with the purpose and mood of the statements, whether a question is being asked, an answer being given, a request being made, an opinion being offered, or information given.

3. The *textual function* dictates the necessity for continuity and coherence between the current and previously stated expressions. This function is concerned with the theme of the conversation, what is known, and what is newly expressed.

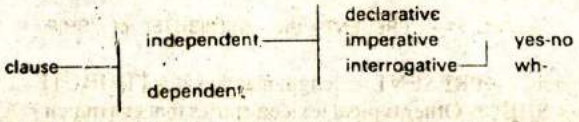
Halliday proposed a model of language which consisted of four basic categories.

Language units. A hierarchy for sentences based on the sentence, clause, phrase group, word, and morpheme.

Role structure of units. A unit consists of one or more units of lower rank based on its role, such as subject, predicate, complement, or adjunct.

Classification of units. Units are classified by the role they play at the next higher level. For example, the verbal serves as the predicate, the nominal serves as the subject or complement, and so on.

System constraints. These are constraints in combining component features. For example, the network structure given below depicts the constraints in an interpretation.



Given these few principles, it is possible to build a grammar which combines much semantic information with the syntactic. Many of the ideas from systemic grammars were used in the successful system SHRDLU developed by Terry Winograd (1972). This system is described later in the chapter.

Semantic Grammars

Semantic grammars encode semantic information into a syntactic grammar. They use context-free rewrite rules with nonterminal semantic constituents. The constituents are categories, or metasymbols such as attribute, object, present (as in display), and ship, rather than NP, VP, N, V, and so on. This approach greatly restricts the range of sentences which can be generated and requires a large number of rewrite rules.

Semantic grammars have proven to be successful in limited applications including LIFER, a data base query system distributed by the Navy which is accessible through ARPANET (Hendrix et al., 1978), and a tutorial system named SOPHIE which is used to teach the debugging of circuit faults. Rewrite rules in these systems essentially take the forms

S → What is <OUTPUT-PROPERTY> of <CIRCUIT-PART>?
OUTPUT-PROPERTY → the <OUTPUT-PROP>
OUTPUT-PROPERTY → < OUTPUT-PROP>
CIRCUIT-PART → C23
CIRCUIT-PART → D12
OUTPUT-PROP → voltage
OUTPUT-PROP → current

In the LIFER system, there are rules to handle numerous forms of wh-queries such as

What is the name and location of the carrier nearest to New York
Who commands the Kennedy

Which convoy escorts have inoperative radar units
When will they be repaired
What Soviet ship has hull number 820

These sentences are analyzed and words matched to metasymbols contained in lexicon entries. For example, the input statement "Print the length of the Enterprise" would fit with the LIFER top grammar rule (L.T.G.) of the form

<L.T.G.> → <PRESENT> the <ATTRIBUTE> of <SHIP>

where print matches <PRESENT>, length matches <ATTRIBUTE>, and the Enterprise matches <SHIP>. Other typical lexicon entries that can match <ATTRIBUTE> include CLASS, COMMANDER, FUEL, TYPE, BEAM, LENGTH, and so on.

LIFER can also accommodate elliptical (incomplete) inputs. Given the query "What is the length of the Kennedy?" a subsequent query consisting of the abbreviated form "of the Enterprise?" will elicit a proper response (see also the third and fourth example queries above).

Semantic grammars are suitable for use in systems with restricted grammars since computation is limited. They become unwieldy when used with general purpose language understanding systems, however.

12.4 BASIC PARSING TECHNIQUES

Before the meaning of a sentence can be determined, the meanings of its constituent parts must be established. This requires a knowledge of the structure of the sentence, the meanings of individual words and how the words modify each other. The process of determining the syntactical structure of a sentence is known as parsing.

Parsing is the process of analyzing a sentence by taking it apart word-by-word and determining its structure from its constituent parts and subparts. The structure of a sentence can be represented with a syntactic tree or a list as described in the previous section. The parsing process is basically the inverse of the sentence generation process since it involves finding a grammatical sentence structure from an input string. When given an input string, the lexical parts or terms (root words) must first be identified by type, and then the role they play in a sentence must be determined. These parts can then be combined successively into larger units until a complete tree structure has been completed.

To determine the meaning of a word, a parser must have access to a lexicon. When the parser selects a word from the input stream it locates the word in the lexicon and obtains the word's possible function and other features, including semantic information. This information is then used in building a tree or other representation structure. The general parsing process is illustrated in Figure 12.5.

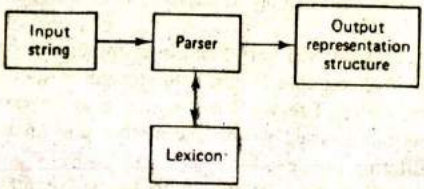


Figure 12.5 Parsing an input to create an output structure.

The Lexicon

A lexicon is a dictionary of words (usually morphemes or root words together with their derivatives), where each word contains some syntactic, semantic, and possibly some pragmatic information. The information in the lexicon is needed to help determine the function and meanings of the words in a sentence. Each entry in a lexicon will contain a root word called the head. Different derivatives of the word, if any, will also be given, and the roles it can play in a sentence (e.g. its part of speech and sense). A fragment of a simplified lexicon is illustrated in Figure 12.6.

The abbreviations 1s, 2s, . . . , 3p in Figure 12.6 stand for first person singular, second person singular, . . . , third person plural, respectively. Note that some words have more than one type such as can which is both a noun and a verb, and orange which is both an adjective and a noun. A lexicon may also be organized to contain separate entries for words with more than one function by giving them separate identities, can1 and can2. Alternatively, the entries in a lexicon could be grouped and given by word category (by articles, nouns, pronouns, verbs,

Word	Type	Features
a	Determiner	{3s}
be	Verb	Trans: intransitive
boy	Noun	{3s}
can	Noun	{1s, 2s, 3s, 1p, 2p, 3p}
	Verb	Trans: intransitive
carried	Verb	Form: past, past participle
.	.	.
.	.	.
orange	Adjective	
	Noun	{3s}
the	Determiner	{3s, 3p}
to	Preposition	
we	Pronoun	{1p}
		Case: subjective
yellow	Adjective	

Figure 12.6 Typical entries in a lexicon

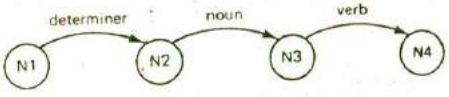
and so on), and all words contained within the lexicon listed within the categories to which they belong.

The organization and entries of a lexicon will vary from one implementation to another, but they are usually made up of variable length data structures such as lists or records arranged in alphabetical order. The word order may also be given in terms of usage frequency so that frequently used words like a, the, and an will appear at the beginning of the list facilitating the search.

Access to the words may be facilitated by indexing, with binary searches, hashing, or combinations of these methods. A lexicon may also be partitioned to contain a base lexicon set of general, frequently used words and domain specific components of words.

Transition Networks

Transition networks are another popular method used to represent formal and natural language structures. They are based on the application of directed graphs (digraphs) and finite state automata. A transition network consists of a number of nodes and labeled arcs. The nodes represent different states in traversing a sentence, and the arcs represent rules or test conditions required to make the transition from one state to the next. A path through a transition network corresponds to a permissible sequence of word types for a given grammar. Thus, if a transition network can be successfully traversed, it will have recognized a permissible sentence structure. For example, a network used to recognize a sentence consisting of a determiner, a noun and a verb ("The child runs") would be represented by the three-node graph as follows.



Starting at node N1, the transition from node N1 to N2 will be made if a determiner is the first input word found. If successful, state N2 is entered. The transition from N2 to N3 can then be made if a noun is found next. The final transition (from N3 to N4) will be made if the last word is a verb. If the three-word category sequence is not found, the parse fails. Clearly, this type of network is very limited since it will only recognize simple sentences of the form DET N V.

The utility of a network such as this could be increased if more than a single choice were permitted at some of the nodes. For example, if several arcs were constructed between nodes N1 and N2 where each arc represented a different noun phrase, the number of permissible sentence types would be increased substantially. Individual arcs could be a noun, a pronoun, a determiner followed by a noun, a determiner followed by an adjective followed by a noun, or some other type of noun phrase which we wish the parser to be capable of recognizing. These alternatives are depicted in Figure 12.7.

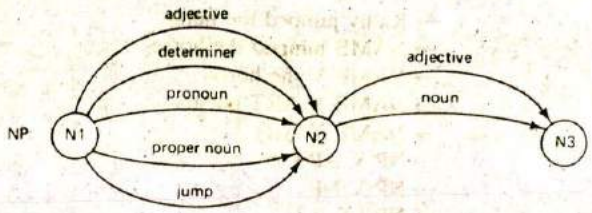


Figure 12.7 A noun phrase segment of a transition network.

To move from state N1 to N2 in this transition network, it is necessray to first find an adjective, a determiner, a pronoun, a proper noun, or none of these by "jumping" directly to N2. This network extends the possible types of sentences that can be recognized substantially over the simple network given above. For example, it will recognize noun phrases having forms such as

Big white fluffy clouds
Our bright children
A large beautiful white flower
Large green leaves
Buildings
Boston's best seafood restaurants

Top-Down versus Botton-up Parsing

Parsers may be designed to process a sentence using either a top-down or a bottom-up approach. A top-down parser begins by hypothesizing a sentence (the symbol S) and successively predicting lower level constituents until individual preterminal symbols are written. These are then replaced by the input sentence words which match the terminal categories. For example, a possible top-down parse of the sentence "Kathy jumped the horse" would be given by

S → NP VP
→ NAME VP
→ Kathy VP
→ Kathy V NP
→ Kathy jumped NP
→ Kathy jumped ART N
→ Kathy jumped the N
→ Kathy jumped the horse

A bottom-up parse, on the other hand, begins with the actual words appearing in the sentence and is, therefore, data driven. A possible bottom-up parse of the same sentence might proceed as follows.

→ Kathy jumped the horse
→ NAME jumped the horse
→ NAME V the horse
→ NAME V ART horse
→ NAME V ART N
→ NP V ART N
→ NP V NP
→ NP VP
→ S

Words in the input sentence are replaced with their syntactic categories and those in turn are replaced by constitutents of the same or smaller size until S has been rewritten or until failure occurs.

Deterministic versus Nondeterministic Parsers

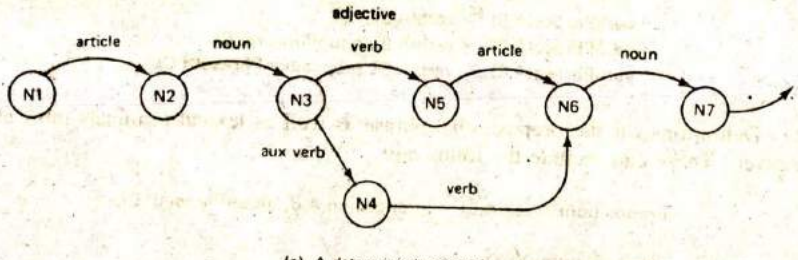
Parsers may also be classified as deterministic or nondeterministic depending on the parsing strategy employed. A deterministic parser permits only one choice (arc) for each word category. Thus, each arc will have a different test condition. Consequently, if an incorrect test choice is accepted from some state, the parse will fail since the parser cannot backtrack to an alternative choice. This may occur, for example, when a word satisfies more than one category such as a noun and a verb or an adjective, noun, and verb. Clearly, in deterministic parsers, care must be taken to make correct test choices at each stage of the parsing. This can be facilitated with a look-ahead feature which checks the categories of one or more subsequent words in the input sentence before deciding in which category to place the current word. Some researchers prefer to use deterministic parsing since they feel it more closely models the way humans parse input sentences.

Nondeterministic parsers permit different arcs to be labeled with the same test. Consequently, the next test from any given state may not be uniquely determined by the state and the current input word. The parser must guess at the proper constituent and then backtrack if the guess is later proven to be wrong. This will require saving more than one potential structure during parts of the network traversal. Examples of both deterministic and nondeterministic parsers are presented in Figure 12.8.

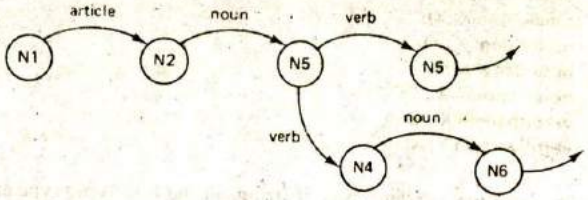
Suppose the following sentence is given to a deterministic parser with the grammar given by the network of Figure 12.8(a): "The strong bear the loads." If the parser chose to recognize strong as an adjective and bear as a noun, the parse would fail, since there is no verb following bear. A nondeterministic parser, on the other hand, would simply recover by backtracking when failure was detected and then taking another arc which accepted strong as a noun.

Example of a Simple Parser in Prolog

The reader may have noticed the close similarity between rewrite rules and Horn clauses, especially when the Horn clauses are written in the form of PROLOG



(a) A deterministic network



(b) A nondeterministic network

Figure 12.8 Deterministic and nondeterministic networks.

rules. This similarity makes it a straightforward task to write parsers in a language like PROLOG. For example, the grammar rule that states that S is a sentence if it is a noun phrase followed by a verb phrase (S → NP VP) may be written in PROLOG as

```
sentence(A,C) :-nounPhrase(A,B), verbPhrase(B,C)
```

The variables A, B, and C in this statement represent lists of words. The argument A is the whole list of words to be tested as a sentence, and C is the list of remaining words, if any. Similar assumptions hold for A, B, and C in the noun and verb phrase conditions respectively.

Rule definitions which rewrite the noun phrases and verb phrases must also be defined. Thus, an NP may be defined with statements such as the following:

```
nounPhrase(A,C) :=article(A,B), noun(B,C).
nounPhrase(A,B) :=noun(A,B).
```

Like the above rule, these rules state that (1) a noun phrase can be either an article which consists of a list A and remaining list B (if any) and a noun which is a list B and remaining list C or (2) a noun consisting of the list A with remaining list B (if any). Similarly, a verb phrase may be defined with rules like the following:

```
verbPhrase(A,B) :- verb(A,B).
verbPhrase(A,C) :- verb(A,B), nounPhrase(B,C).
verbPhrase(A,C) :- verb(A,B), prepositionPhrase(B,C).
```

Definitions for the prepositional phrase as well as lexical terminals must also be given. These can include the following:

```
prepositionPhrase(A,C) :- preposition(A,B), nounPhrase(B,C).

preposition([at|X],X).
article([a|X],X).
article([the|X],X).
noun([dog|X],X).
noun([cow|X],X).
noun([moon|X],X).
verb([barked|X],X).
verb([winked|X],X).
```

With this simple parser we can determine if strings of the following type are grammatically correct.

The dog barked at the cow.
The moon winked at the dog.
A cow barked at a moon.

To do so, we must enter sentence queries as lists such as the following for the PROLOG interpreter:

```
?- sentence([the,dog,barked,at,the,moon],X).
X = [ ]
?- sentence([barked,a,moon,dog,the],X).
no
```

Since the remainder of the sentence bound to X is the empty set, it is recognized as correct. The second sentence failed since it could not instantiate with the correct constituent parts.

Of course, for a parser to be of much practical use, other constituents and a great many more words should be defined. The example illustrates the utility of using PROLOG as a basic parser.

Recursive Transition Networks

The simple networks described above are not powerful enough to recognize the variety of sentences a human language system could be expected to cope with. In fact, they fail to recognize all languages that can be generated by a context-free

grammar. Other extensions are needed to accept a wider range of sentences but still avoid the necessity for large complex networks. We can achieve such extensions by labeling some arcs as a separate network state (such as an NP) and then constructing a subnetwork which recognizes the different noun phrases required. In this way, a single subnetwork for an NP can be called from several places in a sentence. Similar arcs can be labeled for other sentence constituents including VP, PP (prepositional phrases) and others. With these additions, complex sentences having embedded phrases can be parsed with relatively simple networks. This leads directly to the notion of using recursion in a network.

A recursive transition network (RTN) is a transition network which permits arc labels to refer to other networks (including the network's own name), and they in turn may refer back to the referring network rather than just permitting word categories used previously. For example, an RTN described by William Woods (1970) is illustrated in Figure 12.9 where the main network calls two subnetworks and an NP and PP network as illustrated in 12.9(b) and (c).

The top network in the figure is the top level (sentence) network, and the lower level networks are for NP and PP arc states. The arcs corresponding to these states will be traversed only if the corresponding subnetworks (b) or (c) are successfully traversed.

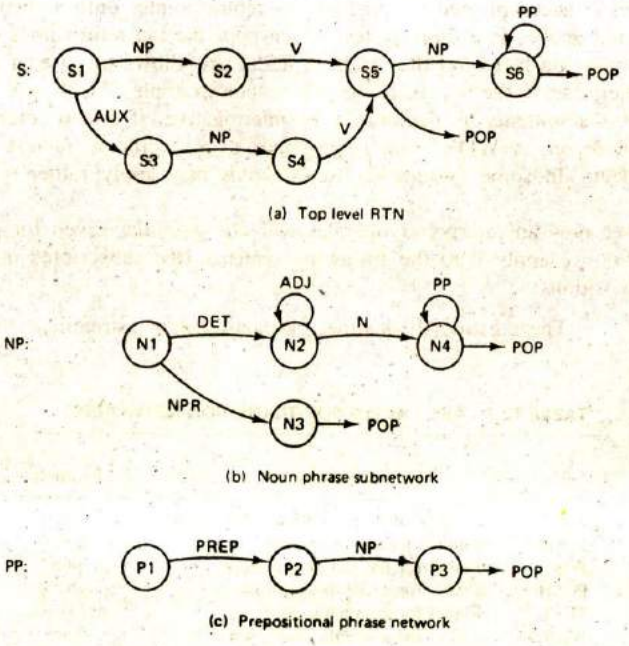


Figure 12.9 Recursive transition network.

In traversing a network, it is customary to test the arcs in a clockwise order. Thus, in the top level RTN, the NP arc will be called first. If this arc fails, the arc labeled AUX will be tested next.

During the traversal of an RTN, a record must be maintained of the word position in the input sentence and the current state or node position and return nodes to be used as return points when control has been transferred to a lower level network. This information can be maintained as a triple like POS CND RLIST where POS is the current input word position, CND is the current node, and RLIST is the list of return points. The RLIST can be maintained as a stack data structure.

In Figure 12.9, the arc named POP is used as a dummy arc to signal the successful completion of the subnetwork and a return to the node following the arc from which it was called. Some other arc types that will be useful in what follows are summarized in Table 12.1.

The CAT arc represents a test for a specific word category such as a noun or a verb. When the input word is of the specified category, the CAT test succeeds and the input pointer is advanced to the next word. The JUMP arc may be traversed without satisfying any test condition in which case the word pointer is not advanced when a JUMP arc is traversed. An arc labeled with a state, such as NP or PP, is defined as a PUSH arc. This arc initiates a call to another network with the indicated state (such as an NP or PP). When a PUSH arc is taken, a return state must be saved. This is accomplished by pushing the return pointer onto a stack. The POP arc, as noted above, is a dummy test which pops the top return node pointer that was previously pushed onto the stack. A TEST arc allows the use of an arbitrary test to determine if the arc is to be taken. For example, TEST can be used to determine if a sentence is declarative or interrogative, if one or more negatives occur, and so on. A WORD arc corresponds to a specific word test such as to, from, and at. (In some systems a list of words may apply rather than a single word.)

To see how an interpreter operates with the grammar given for the RTN of Figure 12.6, we apply it to the following sentence (the subscripted numbers give the word positions):

$$_1\text{The}_2\text{big}_3\text{tree}_4\text{shades}_5\text{the}_6\ \text{old}_7\text{house}_8\text{by}_9\text{the}_{10}\text{stream}_{11}.$$

TABLE 12.1 ARC LABELS FOR TRANSITION NETWORKS

Type of arc	Purpose of arc	Example
CAT	a test label for the current word category	V
JUMP	requires no test to succeed	jump
POP	a label for the end of a network	pop
PUSH	a label for a call to a network	NP
TEST	a label for an arbitrary test	negatives
WORD	a label for a specific word type	from

Starting with CND set to S1, POS set to 1, and RLIST set to nil, the first arc test (NP) would be completed. Since this test is for a state, the parser would PUSH the return node S2 onto RLIST, set CND to N1, and call the NP network. Trying the first test DET (a CAT test) in the NP network, a match would be found with word position 1. This would result in CND being updated to N2 and POS to position 2. The next word (big) satisfies the ADJ test causing CND to be updated to N2 again, and POS to be updated to position 3. The ADJ test is then repeated for the word tree, but it fails. Hence, the arc test for N is made next with no change made to POS and CND. This time the test succeeds resulting in updates of N4 to CND and position 4 to POS. The next test is the POP which signals a successful completion of the NP network and causes the return node (S1) to be retrieved from the RLIST stack and CND to be updated with S2. POP does not cause an advance in the word position POS.

The only possible test from S2 is for category V which succeeds on the word "shades" with resultant updates of S5 to CND and 5 to POS. At S5, the only possible test is the NP. This again invokes a call to the lower level NP network which is traversed successfully with the noun phrase "the old house." After a return to the main network, CND is set to S6 and POS is set to position 6. At this point, the lower PP network is called with CND being set to P1 and S6 pushed onto RLIST. From P1, the CAT test for PREP passes with CND being set to P2 and POS being set to 9. NP is then called with CND being set to N1 and P2 being pushed onto RLIST. As before, the NP network is traversed with the noun phrase "the stream" resulting in a POS value of 11, P3 being popped from RLIST and a return to that node. The test at P3 (POP) results in S6 being popped from RLIST and a return to the S6 node. Finally, the POP test at N6, together with the period at position 11 results in a successful traversal and acceptance of the sentence.

During a network traversal, a parse can fail if (1) the end of the input sentence (a period) has been reached when the test from the CND node value is not a terminal (POP) value or (2) if a word in the input sentence fails to satisfy any of the available arc tests from some node in the network.

The number of sentences accepted by an RTN can be extended if backtracking is permitted when a failure occurs. This requires that states having alternative transitions be remembered until the parse progresses past possible failure points. In this way, if a failure occurs at some point, the interpreter can backtrack and try alternative paths. The disadvantage with this approach is that parts of a sentence may be parsed more than one time resulting in excessive computations.

Augmented Transition Networks

The networks considered so far are not very useful for language understanding. They have only been capable of accepting or rejecting a sentence based on the grammar and syntax of the sentence. To be more useful, an interpreter must be able to build structures which will ultimately be used to create the required knowledge entities for an AI system. Furthermore, the resulting data structures should contain

more information than just the syntactic information dictated by the grammar alone. Semantic information should also be included. For example, a number of sentence features can also be established and recorded, such as the subject NP, the object NP, the subject-verb number agreement, the mood (declarative or interrogative), tense, and so on. This means that additional tests must be performed to determine the possible semantics a sentence may have. Without these additional tests, much ambiguity will still be present and incorrect or meaningless sentences accepted.

We can achieve the additional capabilities required by augmenting an RTN with the ability to perform additional tests and store immediate results as a sentence is being parsed. When an RTN is given these additional features, it is called an augmented transition network or ATN.

When building a representation structure, an ATN uses a number of different registers as temporary storage to hold the different sentence constituents. Thus, one set of registers would be used for an NP network, one for a PP network, one for a V, and so on. Using the register contents, an ATN builds a partial structural description of the sentence as it moves from state to state in the network. These registers provide temporary storage which is easily modified, switched, or discarded until the final sentence structure is constructed. The registers also hold flags and other indicators used in conjunction with some arcs. When a partial structure has been stored in registers and a failure occurs, the interpreter can clear the registers, backtrack, and start a new set of tests. At the end of a successful parse, the contents of the registers are combined to form the final sentence data structure required for output.

An ATN Specification Language

A specification language developed by Woods (1970, 1986) for ATNs takes the form of an extended context-free grammar. This language is given in Figure 12.10 where the vertical bar indicates alternative choices for a construction and the * (Kleene star) signifies repeatable (zero or more) elements. All nonterminals are enclosed in angle brackets. Some of the capitalized words appearing in the language were defined earlier as arc tests and actions. The other words in uppercase correspond to functions which perform many of the tasks related to the construction of the structure using the registers.

The specification language is read the same as rewrite rules. Thus, it specifies that a transition network is composed of a list of arc sets, where each arc set is in turn a list with first element being a state name and the remaining elements being arcs which emanate from that state. An arc can be any of the forms CAT, JUMP, PUSH, TEST, WORD or POP. For example, as noted earlier, the TEST arc corresponds to an arbitrary test which determines whether the arc is to be traversed or not. Note that a sequence of actions is associated with the arc tests. These actions are executed during the arc traversals. They are used to build pieces of structures such as a tree or a list. The terminal action of any arc specifies the state to which control is passed to complete the transition.

```
<transition net> → (<arc set><arc set>*)
<arc set> → (<state><arc>*)
<arc> → (CAT <category name><test><action>*<term act>)|
        (PUSH <state><test><action>*<term act>)|
        (TST (arbitrary label><test><action>*<term act>)|
        (POP <form><test>)
<action> → (SETR <register><form>)|
           (SENDR <register><form>)|
           (LIFTR <register><form>)
<term act> → (TO <state>)|
             (JUMP <state>)
<form> → (GETR <register>)|
         @|
         (GETF <feature>)|
         (BUILDQ <fragment><register>*)|
         (LIST <form>*)|
         (APPEND <form><form>)|
         (QUOTE <arbitrary structure>)
```

Figure 12.10 A specification language for ATNs.

Among other things, an action can be any of the three function forms SETR, SENDR, and LIFTR which cause the indicated register values to be set to the value of form. Terminal actions can be either TO or JUMP where TO requires that the input sentence pointer should be advanced, and JUMP requires that the pointer remain fixed and the input word continue to be scanned. Finally, a construction form can be any of the seven alternatives in the bottom group of Figure 12.10, including the symbol @ which is a terminal symbol placeholder for form.

The function SETR causes the contents of the indicated registers to be set equal to the value of the corresponding form. This is done at the current level in the network, while SENDR causes it to be done by sending it to the next lower level of computation. LIFTR returns information to the next higher level of computation. The function GETR returns the value of the indicated register, and GETF returns the value of a specified feature for the current input word. As noted before, the value of @ is usually an input word. The function BUILDQ takes lists from the indicated registers (which represent fragments of a parse tree with marked nodes) and builds the sentence structures.

An ATN network similar to the RTN illustrated in Figure 12.9 is presented in Figure 12.11. Note that the arcs in this network have some of the tests described above. These tests will have the basic forms given in Figure 12.10, together with the indicated actions. The actions include building the final sentence structure which may contain more features than those considered thus far, as well as certain semantic features.

Using the specification language, we can represent this particular network with the constituent abbreviations and functions described above in the form of a LISP program. For example, a partial description of the network is depicted in

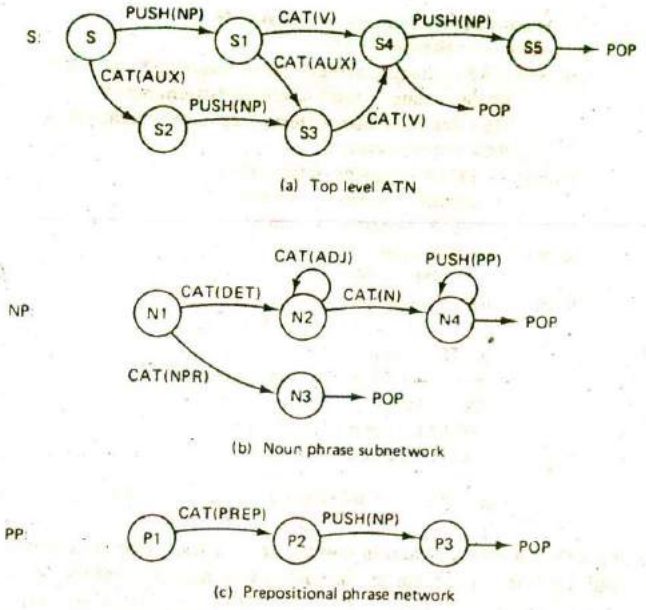


Figure 12.11 Augmented transition network.

Figure 12.12 (where T in the expressions is the equivalent of non-nil or true in LISP).

From the language of Figure 12.12, it can be seen that the ATN begins building a sentence representation in which the first constituent is either type declarative (DCL) or type interrogative (Q for question), depending on whether the first successful test is an NP or AUX, respectively. The next constituent is the subject (SUBJ) NP, and the third is either an auxiliary verb (AUX) or nil. The fourth constituent is a VP. An ATN is traversed much the same as the RTN described above.

An example of its operation will help to demonstrate how the structure is built during a parse of the sentence

"The big dog likes the small boy."

1. Starting with state S, PUSH down a level to the NP network. If an NP is found (T for true), execute lines 2, 3, and 4.

2. In the lower level NP network, the noun phrase "the big dog" is found with successive CAT tests for determiner, adjective, and noun. During these tests, NP registers are set to indicate the word constituents. When the terminal node

```
1. (S/ (PUSH NP/ T
2.       (SETR SUBJ @)
3.       (SETR TYPE (QUOTE DCL))
4.       (TO S1)
5.     (CAT AUX T
6.       (SETR AUX @)
7.       (SETR TYPE (QUOTE Q))
8.       (TO S2)))
9. (S1 (CAT V T
10.      (SETR AUX NIL)
11.      (SETR V @)
12.      (TO S4)))
13.    (CAT AUX T
14.      (SETR AUX @)
15.      (TO S3)))
16. (S2 (PUSH NP/ T
17.      (SETR SUBJ @)
18.      (TO S3)))
19. (S3 (CAT V T
20.      (SETR V @)
21.      (TO S4)))
22. (S4 (POP BUILDQ (S+ + +(VP+)) TYPE SUBJ AUX V) T)
23.    (PUSH NP/ T
24.      (SETR VP BUILDQ (VP (V+) @) V))
25.      (TO S5)))
26. (S5 (POP (BUILDQ (S+ + + +) TYPE SUBJ AUX VP) T)
27.    (PUSH PP/ T
28.      (SETR VP (APPEND (GETR VP) (LIST @)))
29.      (TO S5)))
```

Figure 12.12 An ATN specification language.

(N4) is tested and the PP test subsequently fails, POP is executed and a return of control is made to statement 2.

3. The register SUBJ is set to the value of @ which is the list structure (NP (dog (big) DEF)) returned from the NP registers. DEF signifies that the determiner is definite.

4. In line 3, register TYPE is set to DCL (for declarative).

5. Control is transferred to S1 with the statement TO in line 4 and the input pointer is moved past the noun phrase to the verb "likes."

6. If an auxiliary verb had been found at the beginning of the sentence instead of an NP, control would have been passed to line 5 where statements 5, 7, and 8 would have been executed. This would have resulted in registers AUX and TYPE being set to the values @ and Q respectively.

7. At S1, a category test is made for a V. Since this succeeds (is T), statements 11, 12, and 13 are executed. This results in register AUX being set to nil, and register V being set to the contents of @, to give (V likes). Control is then passed to S4 and the input pointer is moved to the word "the."

8. If the test for V had failed, and an auxiliary verb had been found, statements 14 and 15 would have been executed.

9. Since S4 is a terminal node, a sentence structure can be built there. This will be the case if the end of the sentence has been reached. If so, the BUILDQ function creates a list structure with first element S, followed by the values of the three registers TYPE, SUBJ, AUX, corresponding to the three plus (+) signs. These are then followed with VP and the contents of the V register. For example, with an input sentence of,

The boy can whistle.

the structure (S DCL (NP (boy) DEF) (AUX can) (VP whistle)) would be constructed from the four registers TYPE, SUBJ, AUX, and V.

10. Because more input words remain, the BUILDQ in line 22 is not executed, and control drops to the next line where a push is made to the lower NP network. As before, the NP succeeds with the structure (NP (boy (small) DEF)) being returned as the value of @. Register VP is then set to the list returned by BUILDQ (line 24) which consists of VP followed by the verb phrase and control is passed to S5.

11. Since S5 is a terminal node and the end of the input sentence has been reached, BUILDQ will build the final sentence structure from the TYPE, SUBJ, AUX, and VP register contents. The final structure constructed is

```
(S DCL (NP (dog (big) DEF))
       (VP (V likes)(NP (boy (small) DEF))))
```

The use of recursion, arc tests, and a variety of arc and node combinations give the ATNs the power of a Turing Machine. This means that an ATN can recognize any language that a general purpose computer can recognize. This versatility also makes it possible to build deep sentence structures rather than just structures with surface features only. (Recall that surface features relate to the form of words, phrases, and sentences, whereas deep features relate to the content or meaning of these elements). The ability to build deep structures requires that other appropriate tests be included to check pronoun references, tense, number agreement, and other features.

Because of their power and versatility, ATNs have become popular as a model for general purpose parsers. They have been used successfully in a number of natural language systems as well as front ends for databases and expert systems.

12.5 SEMANTIC ANALYSIS AND REPRESENTATION STRUCTURES

We have now seen how the structure of a complex sentence can be determined and how a representation of that structure can be constructed using different types of parsers. In particular, it should now be clear how an ATN can be used to build structures for different grammars, like those described in Section 12.3. But, we have not yet explained how the final semantic knowledge structures are created to satisfy the requirements of a knowledge base used to represent some particular world model. Experience has shown the semantic interpretation to be the most difficult stage in the transformation process.

As an example of some of the difficulties encountered in extracting the full intended meaning of some utterances, consider the following situation.

> It turned into a black day. In his haste to catch the flight, he backed over Tom's bicycle. He should never have left it there. It was damaged beyond repair. That caused the tailpipe to break. It would be impossible to make it now. . . . It was all because of that late movie. He would be heartbroken when he found out about it.

Although a car was never explicitly mentioned, it must be assumed that a car was the object which was backed over Tom's bicycle. A program must be able to infer this. The "black day" metaphor also requires some inference. Days are not usually referred to by color. And sorting out the pronoun references can also be an onerous task for a program. Of the seven uses of it, two refer to the bicycle, two to the flight, two refer to the situation in general, and one to the status of the day. There are also four uses of he referring to two different people and a that which refers to the accident in general. The placement of the pronouns is almost at random making it difficult to give any rule of association. Words that point back or refer to people, places, objects, events, times, and so on that occurred before, are called anaphors. Their interpretation may require the use of heuristics, syntactic and semantic constraints, inference, and other forms of object analysis within the discourse content.

This example should demonstrate again that language cannot be separated from intelligence and reasoning. To fully understand the above situation requires that a program be able to reason about people's goals, beliefs, motives, and facts about the world in general.

The semantic structures constructed from utterances such as the above, must account for all aspects of meaning in what is known as the domain, context, and the task. The *domain* refers to the knowledge that is part of the world model the system knows about. This includes object descriptions, relationships, and other relevant concepts. The *context* relates to previous expressions, the setting and time of the utterances, and the beliefs, desires, and intentions of the speakers. A *task* is part of the service the system offers, such as retrieving information from a data base, providing expert advice, or performing a language translation. The domain, context, and task are what we have loosely referred to before as semantics, pragmatics, and world knowledge.

Semantic interpretations require that utterances be transformed into coherent expressions in the form of FOPL clauses, associative networks, frames, or script-like structures that can be manipulated by the understanding program. There are a number of different approaches to the transformation problem. The approach we have been following up to this point is one in which the transformation is made in stages. In the first stage, a syntactic analysis is performed and a tree-like structure is produced using a parser. This stage is followed by use of a semantic analyzer to produce either an intermediate or final semantic structure.

Another approach is to transform the sentences directly into the target structures with little syntactical analysis. Such approaches typically depend on the use of constraints given by semantic grammars or place strong reliance on the use of key words to extract meaning.

Between these two extremes, are approaches which perform syntactic and semantic analyses concurrently, using semantic information to guide the parse, and the structure learned through the syntactical analysis is used to help determine meaning.

Another dimension related to semantic interpretation is the approach taken in extracting the meaning of an expression. (1) whether it can or should be derived by paraphrasing input utterances and transforming them into structures containing a few generic terms or (2) whether meanings are best derived by composing the meanings of clauses and larger units from the meanings of their constituent parts. These two methods of approach are closely tied to the form of the target semantic structures.

The first of these approaches we call unit or *lexical semantics* to emphasize the role played by the special primitive words used to represent the meanings of all expressions. In this approach the meaning is constructed through a restatement of the expression in terms of linked primitive generic words such as those used in Shank's conceptual dependency theory (Chapter 7).

The second approach is called *compositional semantics* since the meaning of an expression is derived from the meanings ascribed to the constituent parts. The structures created through this approach are usually characterized as logical formulae in some calculus such as FOPL or an extended FOPL.

Lexical Semantics Approaches

The semantic grammars described in Section 12.2 are one form of approach based on the use of lexical semantics. With this approach, input sentences are transformed through the use of domain dependent semantic rewrite rules which create the target knowledge structures. A second example of an informal lexical-semantic approach is one which uses conceptual dependency theory. Conceptual dependency structures provide a form of linked knowledge that can be used in larger structures such as scenes and scripts.

The construction of conceptual dependency structures is accomplished without performing any direct syntactic analysis. Making the jump between utterances and

ACTOR: (a PP with animate attributes)
OBJECT: (a PP)
ACTION: (one of the primitive acts with tense)
DIRECTION: (from-to direction of the action)
INSTRUMENT: (object with which the act is performed)
LOCATION: (event location information)
TIME: (time of the event information)

Figure 12.13 Conceptual dependency structure.

these structures requires that more information be contained in the lexicon. The lexicon entries must include word sense and other information which relate the words to a number of primitive semantic categories as well as some syntactic information.

Recall from Chapter 7 that conceptualizations are either events or object states. Event structures include objects and their attributes, picture producers (PPs) or actors, actions, direction of action (to or from) and sometimes instruments that participate in the actions, and the location and time of the event. These items are collected together in a slot-filler structure as depicted in Figure 12.13.

Verbs in the input string are a dominant factor in building conceptual dependency structures because they denote the event action or state. Consequently, lexicon entries for verbs will be more extensive than other entry types. They will contain all possible senses, tense, and other information. Each verb maps to one of the primitive actions: ATRANS, ATTEND, CONC, EXPEL, GRASP, INGEST, MBUILD, MOVE, MTRANS, PROPEL, PTRANS, and SPEAK. Each primitive action will also have an associated tense: past, present, future, conditional, continuous, interrogative, end, negation, start, and timeless.

The basic process followed in building conceptual dependency structures is simply the three steps listed below.

1. Obtain the next lexical item (a word or phrase).
2. Access the lexical entry for the item and obtain the associated tests and actions.
3. Perform the specified actions given with the entry.

Three types of tests are performed in Step 2.

1. If a certain lexical entry is found, the indicated action is performed. This corresponds to true (non-nil in LISP).
2. Specific word orderings are checked as the structure is being built and actions initiated as a result of the orderings. For example, if a PP follows the last word parsed, action is initiated to fill the object slot.
3. Checks are made for specific words or phrases and, if found, specified actions taken. For example, if an intransitive verb such as listen is found, an action

would be initiated to look for associated words which complete the phrase beginning with to or for.

For the above tests, there are four types of actions taken.

1. Adding additional structure to a partially built conceptual dependency.
2. Filling a slot with a substructure.
3. Activating another action.
4. Deactivating an action.

These actions build up the conceptual dependency structure as the input string is parsed. For example, the action taken for a verb like drank would be to build a substructure for the primitive action INGEST with unfilled slots for ACTOR, OBJECT, and TENSE.

```
(INGEST (ACTOR nil)
        (OBJECT nil)
        (TENSE past))
```

Subsequent words in the input string would initiate actions to add to this structure and fill in the empty ACTOR and OBJECT slots. Thus, a simple sentence like

The boy drank a soda

would be transformed through a series of test and action steps to produce a structure such as the following.

```
(INGEST (ACTOR (PP (NAME boy) (CLASS PHYS-OBJ)
               (TYPE ANIMATE) (REF DEF)))
        (OBJECT (PP (NAME soda) (CLASS PHYS-OBJ)
                (TYPE INANIMATE) (REF INDEF)))
        (TENSE PAST))
```

Compositional Semantics Approaches

In the compositional semantics approach, the meaning of an expression is derived from the meanings of the parts of the expression. The target knowledge structures constructed in this approach are typically logic expressions such as the formulas of FOPL. The LUNAR system developed by Woods (1978) uses this approach. The input strings are first parsed using an ATN from which a syntactic tree is output. This becomes the input to a semantic interpreter which interprets the meaning of the syntactic tree and creates the semantic representations.

As an example, suppose the following declaration is submitted to LUNAR.

Sample24 contains silicon

This would be parsed, and the following tree structure would be output from the ATN:

```
(S DCL
  (NP   (N (Sample24)))
  (AUX (TENSE (PRESENT)))
  (VP   (V (contain))
        (NP (N (silicon))))
```

Using this structure, the semantic interpreter would produce the predicate clause

```
(CONTAIN sample24 silicon)
```

which has the FOPL meaning you would expect.

The interpreter used in LUNAR is driven by semantic pattern → action interpretation rules. The rule that builds the CONTAIN predicate is selected whenever the input tree has a verb of the form have or contain and a sample as the subject and either chemical element, isotope, or oxide as an object. The action of the rule states that such a sentence is to be interpreted as an instance of the schema (CONTAIN *x y*) with *x* and *y* being replaced by the ATN's interpretation of subject noun phrase and object respectively.

LUNAR is also capable of performing quantification of variables in expressions. The quantifiers are an elaboration of those used in FOPL. They include the standard existential and universal quantifiers "for every" and "for some," as well as others such as "for the," "exactly," "the single," "more than," "at least," and so on. For example, an expression with universal quantification would appear as

```
(For Every X (SEQ samples):CONTAIN X Overall silicon)
```

12.6 NATURAL LANGUAGE GENERATION

It is sometimes claimed that language generation is the exact inverse of language understanding. While it is true the two processes have many differences, it is an over simplification to claim that they are exact opposites.

The generation of natural language is more difficult than understanding it, since a system must not only decide what to say, but how the utterances should be stated. A generation system must decide which form is better (active or passive), which words and structures best express the intent, and when to say what. To

produce expressions that are natural and close to humans requires more than rules of syntax, semantics, and discourse. In general, it requires that a coherent plan be developed to carry out multiple goals. A great deal of sophistication goes into the simplest types of utterances when they are intended to convey different shades of meanings and emotions. A participant in a dialog must reason about a hearer's understanding and his or her knowledge and goals. During the dialog, the system must maintain proper focus and formulate expressions that either query, explain, direct, lead or just follow the conversation as appropriate.

The study of language generation falls naturally into three areas: (1) the determination of content, (2) formulating and developing a text utterance plan, and (3) achieving a realization of the desired utterances.

Content determination is concerned with what details to include in an explanation, a request, a question or argument in order to convey the meanings set forth by the goals of the speaker. This means the speaker must know what the hearer already knows, what the hearer needs to know, and what the hearer wants to know. These topics are related to the domain, task, and discourse context described above. *Text planning* is the process of organizing the content to be communicated so as to best achieve the goals of the speaker. *Realization* is the process of mapping the organized content to actual text. This requires that specific words and phrases be chosen and formulated into a syntactic structure.

Until about 1980, not much work had been done beyond single sentence generation. Understanding and generation was performed with a single piece of isolated text without much regard given to context and consideration of the hearer. Following this early work, a few comprehensive systems were developed. To complete this section, we describe the basic ideas behind two of these systems. They take different approaches to those taken by the lexical and compositional semantics understanding described in the previous section.

Language Planning and Generation with KAMP

KAMP is a knowledge and modalities planner developed for the generation of natural language text. Developed by Douglas Appelt (1985), KAMP simulates the behavior of an expert robot named Rob (a terminal) assisting John (a person) in the disassembly and repair of air compressors.

KAMP uses a planner and a data base of knowledge in (modal) logical form. The knowledge includes domain knowledge, world knowledge, linguistic knowledge, and knowledge about the hearer. A description of actions and action summaries are available to the planner. Given a goal, the planner uses heuristics to build and refine a plan in the form of a procedural network. Other procedures act as critics of the plans and help to refine them. If a plan is completed, a deduction system is used to prove that the sequence of actions do, in fact, achieve the goal. If the plan fails, the planner must do further searching for a sequence of actions that will work. A completed plan states the knowledge and intentions of the agent, the robot

Rob. This is the first step in producing the output text. The process can be summarized as follows.

Suppose KAMP has determined the immediate goal to be the removal of the compressor pump from the platform.

```
True(¬Attached(pump platform))
```

KAMP first formulates and refines a plan that John adopt Rob's plan to remove the pump from the platform. The first part of Rob's plan suggests a request for John to remove the pump leading to the expression

```
INTENDS(john, REMOVE(pump platform))
```

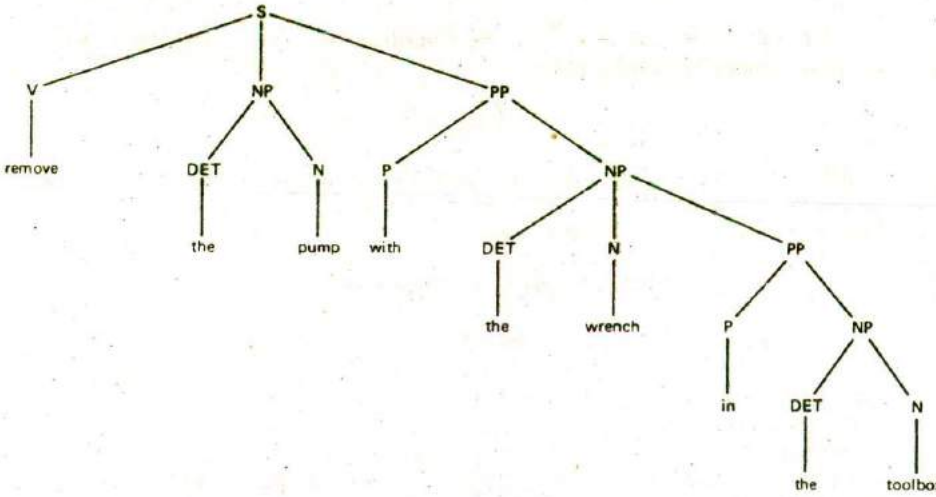
After axioms are used to prove that actions in the initial summary plan are successful, the request is expanded to include details for the pump removal. Rob decides that John will know he is near the platform and that he knows where the toolbox is located, but that he does not know what tool to use. Rob, therefore, determines that John will not need to be told about the platform, but that he must be informed, with an imperative statement, to remove the pump with a wrench in the toolbox.

```
INTENDS(john, REMOVE(pump platform))
LOCATION(john) = LOCATION(platform)

DO(john, REMOVE(pump platform))

DO(rob, REQUEST(john, REMOVE(pump platform)))
DO(rob, COMMAND(john, REMOVE(pump platform)))

DO(rob, INFORM(john (TOOL(wrench))))
DO(rob, INFORM(john LOCATION (wrench) =
                              LOCATION(tool-box)))
```

The next step is for Rob to plan speech acts to realize the request. This requires linguistic knowledge of the structure to use for an imperative request, in this case, that the sentence should have the form V NP (PP)* (recall that * stands for optional repetition). Words to complete the output string are then selected and ordered accordingly.

```
DO(rob REQUEST(john, REMOVE(pump platform)))
DO(rob COMMAND(john, REMOVE(pump platform)))
DO(rob INFORM(john, TOOL(wrench)))
DO(rob INFORM(john, LOCATION(wrench) =
                              LOCATION(tool-box)))
```

This leads to the generation of a sentence with the following tree structure.



The overall process of planning and formulating the final sentence "Remove the pump with the wrench in the toolbox" is very involved and detailed. It requires planning and plan verification for content, selecting the proper structures, selecting senses, mood, tense, the actual words, and a final ordering. All of the steps must be constrained toward the realization of the (possibly multiple) goals set forth. It is truly amazing we accomplish such acts with so little effort.

Generation from Conceptual Dependency Structures

Niel Goldman (Schank et al., 1973) developed a generation component called BABEL which was used as part of several language understanding systems built by Schank and his students (SAM, MARGIE, QUALM, and so on). This component worked in conjunction with an inference component to determine responses to questions about short news and other stories.

Given the general content or primitive event for the response, BABEL selects and builds an appropriate conceptual dependency structure which includes the intended word senses. A modified ATN is then used to generate the actual word string for output.

To determine the proper word sense, BABEL uses a discrimination net. For example, suppose the system is told a story about Joe going into a fast-food restaurant, ordering a sandwich and a soft drink in a can, paying, eating, and then leaving. After the understanding part of the system builds the conceptual dependency and script structures for the story, questions about the events could be posed. If asked what Joe had in the restaurant, BABEL would first need to determine the conceptual

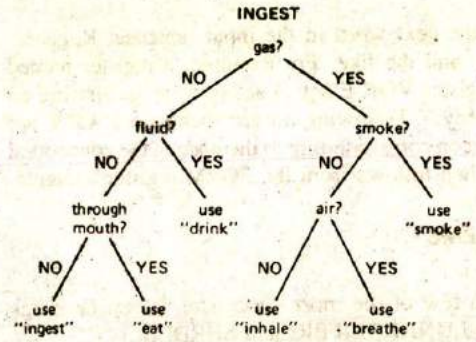
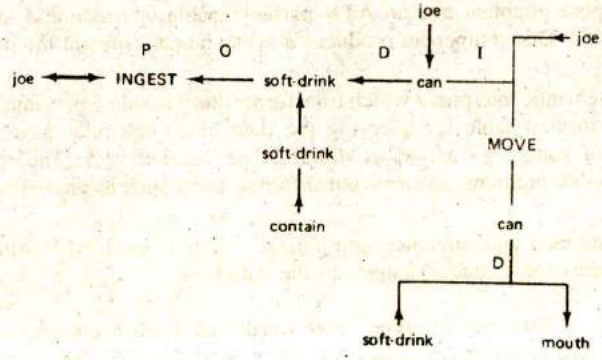


Figure 12.14 Discrimination net for INGEST.

category of the question in order to select the proper conceptual dependency pattern to build. The verb in the query determines the appropriate primitive categories of eat and drink as being INGEST. To determine the correct sense of INGEST as eat and drink a discrimination net like that depicted in Figure 12.14 would be used. A traversal of the discrimination net leads to eat and drink, using the relation from have and sandwich as being taken through the mouth and soft drink as fluid.

Once a conceptual dependency framework has been selected, the appropriate words must be chosen and the slots filled. Functions are used to operate on the net to complete it syntactically to obtain the correct tense, mood, form, and voice. When completed, a modified ATN is then used to transform the conceptual dependency structure into a surface sentence structure for output.

The final conceptual dependency structure passed to the ATN would appear as follows.



An ATN used for text generation differs from one used for analysis. In particular, the registers and arcs must be different. The value of the register contents (denoted as $\bar{a}$ in the previous section) corresponds to a node or arc in the conceptual dependency

(or other type) network rather than the next word in the input sentence. Registers will be present to hold tense, voice, and the like. For example, a register named FORM might be set to past and a register VOICE set to active when generating an active sentence like "Joe bought candy." Following an arc such as a CAT/V arc means there must be a word in the lexicon corresponding to the node in the conceptual dependency. The tense of the word then follows from the FORM register contents.

12.7 NATURAL LANGUAGE SYSTEMS

In this section, we briefly describe a few of the more successful natural language understanding systems. They include LUNAR, LIFER, and SHRDLU.

The LUNAR System

The LUNAR system was designed as a language interface to give geologists direct access to a data base containing information on lunar rock and soil compositions obtained during the NASA Apollo-11 moon landing mission. The design objective was to build a system that could respond to natural queries received from geologists such as

> What is the average concentration of aluminum in high-alkali rocks?
> What is the average of the basalt?
> In which samples has apatite been identified?

LUNAR has three main components:

1. A general purpose grammar and an ATN parser capable of handling a large subset of English. This component produces a syntactic parse tree of the input sentence.
2. A rule-driven semantic interpreter which transforms the syntactic representation into a logical form suitable for querying the data base. The rules have the general form of pattern → action as described in Section 12.5. The rules produce disposable programs to carry out different tasks such as answering a query.
3. A data base retrieval and inference component which is used to determine answers to queries and to make changes to the data base.

The system has a dictionary of some 3500 words, an English grammar and two data bases. One data base contains a table of chemical analyses of about 13,000 entries, and the other contains 10,000 indexed document topics. LUNAR uses a meaning representation language which is an extended form of FOPL. The language uses (1) designators which name objects or classes of objects like nouns, variables, and classes with range quantifiers, (2) propositions that can be true or false, that are connected with logical operators and, or, not, and quantification identifiers.

and (3) commands which carry out specific actions (like TEST which tests the truth value of propositions against given arguments (TEST (CONTAIN sample24 silicon))).

Although never fully implemented, the LUNAR project was considered an operational success since it related to a real world problem in need of a solution. It failed to parse or find the correct semantic interpretation on only about 10% of the questions presented to it.

The LIFER System

LIFER (Language Interface Facility with Ellipsis and Recursion) was described briefly in Section 12.2 under semantic grammars. It was developed by Gary Hendrix (1978) and his associates to be used as a development aid and run-time language interface to other systems such as a data base management system. Among its special features are spelling corrections, processing of elliptical inputs, and the ability of the run-time user to extend the language through the use of paraphrase.

LIFER consists of two major components, a set of interactive functions for language specifications and a parser. The specification functions are used to define an application language as a subset of English that is capable of interacting with existing software. Given the language specification, the parser interprets the language inputs and translates them into appropriate structures that interact with the application software.

In using a semantic grammar, LIFER systems incorporate much semantic information within the syntax. Rather than using categories like NP, VP, N, and V, LIFER uses semantic categories like <SHIP-NAME> and <ATTRIBUTE> which match ship names or attributes. In place of syntactic patterns like NP VP, semantic patterns like What is the <ATTRIBUTE> of <SHIP>? are used. For each such pattern, the language definer supplies an expression with which to compute the interpretations of instances of the pattern. For example, if LIFER were used as the front end for a database query system, the interpretation would be for a database retrieval command.

LIFER has proven to be effective as a front end for a number of systems. The main disadvantage, as noted earlier, is the potentially large number of patterns that may be required for a system which requires many, diverse patterns.

The SHRDLU System

SHRDLU was developed by Terry Winograd as part of his doctoral work at M.I.T. (1972, 1986). The system simulates a simple robot arm that manipulates blocks on a table. During a dialog which is interactive, the system can be asked to manipulate the block objects and build stacks or put things into a box. It can be questioned about the configuration of things on the table, about events that have transpired during the dialog, and even about its reasoning. It can also be told facts which are added to its knowledge base for later reasoning.

The unique aspect of the system is that the meanings of words and phrases are encoded into procedures that are activated by input sentences. Furthermore, the

syntactic and semantic analysis, as well as the reasoning process are more closely integrated.

The system can be roughly divided into four component domains: (1) a syntactic parser which is governed by a large English (systemic type) grammar, (2) a semantic component of programs that interpret the meanings of words and structures, (3) a cognitive deduction component used to examine consequences of facts, carry out commands, and find answers, and (4) an English response generation component. In addition, there is a knowledge base containing blocks world knowledge, and a model of its own reasoning process, used to explain its actions.

Knowledge is represented with FOPL-like statements which give the state of the world at any particular time and procedures for changing and reasoning about the state. For example, the expressions

```
(IS b1 block)
(IS b2 pyramid)
(AT b1 (LOCATION 120 120 0))
(SUPPORT b1 b2)
(CLEARTOP b2)
(MANIPULATE b1)
(IS blue color)
```

contain facts describing that b1 is a block, b2 is a pyramid, and b1 supports b2. There are also procedural expressions to perform different tasks such as clear the top or manipulate an object. The CLEARTOP expression is essentially a procedure that first checks to see if the object X supports an object Y. If so, it goes to GET-RID-OF Y and checks again. Integrating the parts of the understanding process with procedural knowledge has resulted in an efficient and effective understanding system. Of course, the domain of SHRDLU is very limited and closed, greatly simplifying the problem.

12.8 SUMMARY

Understanding and generating human language is a difficult problem. It requires a knowledge of grammar and language, of syntax and semantics, of what people know and believe, their goals, the contextual setting, pragmatics, and world knowledge.

We began this chapter with an overview of topics in linguistics, including sentence types, word functions, and the parts of speech. The different forms of knowledge used in natural language understanding were then presented: phonological, morphological, syntactic, semantic, pragmatic, and world. Three general approaches have been followed in developing natural language systems: keyword and pattern matching, syntactic and semantic directed analysis, and matching real world scenarios. Most of the material in this chapter followed the syntactic and semantic directed approach.

Grammars were formally introduced, and the Chomsky hierarchy was presented. This was followed with a description of structural representations for sentences, the phrase marker. Four additional extended grammars were briefly described. One was the transformational grammars, an extension of generative grammars. Transformational grammars include tree manipulation rules that permit the construction of deeper semantic structures than the generative grammars. Case, semantic, and systemic grammars were given as examples of grammars that are also more semantic oriented than the generative grammars.

Lexicons were described, and the role they play in NL systems given. Basic parsing techniques were examined. We looked at simple transition networks, recursive transition networks, and the versatile ATN. The ATN includes tests and actions as part of the arc components and special registers to help in building syntactic structures. With an ATN, extensive semantic analysis is even possible. We defined top-down, bottom-up, deterministic, and nondeterministic parsing methods, and an example of a simple PROLOG parser was also discussed.

We next looked at the semantic interpretation process and discussed two broad approaches, namely the lexical and compositional semantic approaches. These approaches are also identified with the type of target knowledge structures generated. In the compositional semantics approach, logical forms were generated, whereas in the lexical semantics approach, conceptual dependency or similar network structures are created.

Language generation is approximately the opposite of the understanding analysis process, although more difficult. Not only must a system decide what to say but how to say it. Generation falls naturally into three areas, content determination, text planning, and text realization. Two general approaches were presented. They are like the inverses of the lexical and compositional semantic analysis processes. The KAMP system uses an elaborate planning process to determine what, when, and how to state some concepts. The system simulates a robot giving advice to a human helper in the repair of air compressors. At the other extreme, the BABEL system generates output text from conceptual dependency and script structures.

We concluded the chapter with a look at three systems of somewhat disparate architectures: the LUNAR, LIFER, and SHRDLU systems. These systems typify the state-of-the-art in natural language processing systems.

EXERCISES

12.1. Derive a parse tree for the sentence "Bill loves the frog," where the following rewrite rules are used.

S → NP VP
NP → N
NP → DET N
VP → V NP

DET → the
V → loves
N → bill | frog

12.2. Develop a parse tree for the sentence "Jack slept on the table" using the following rules.

S → NP VP
NP → N
NP → DET N
VP → V PP
PP → PREP NP
N → jack | table
V → slept
DET → the
PREP → on

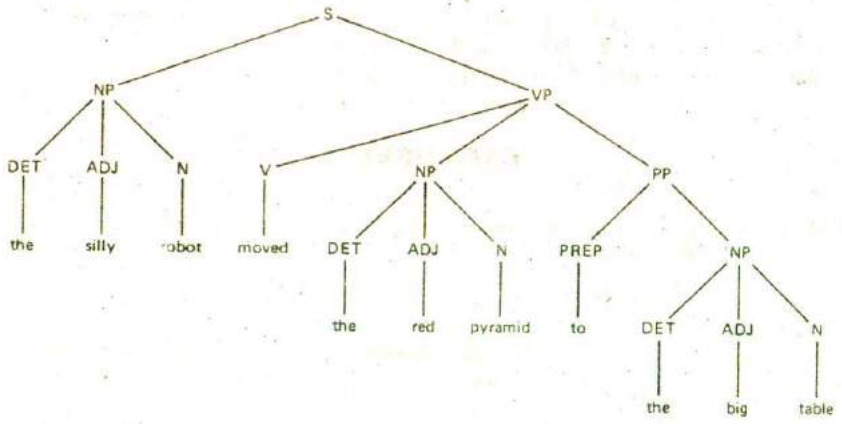
12.3. Give an example of each of the four types 0, 1, 2, and 3 for Chomsky's hierarchy of grammers.

12.4. Modify the grammer of Problem 12.1 to allow the NP (noun phrase) to have zero to many adjectives.

12.5. Explain the main differences between the following three grammars and describe the principal features that could be used to develop specifications for a syntactical recognition program. Consult additional references for more details regarding each grammar.
Chomsky's Transformational Grammar
Fillmore's Case Grammar
Systemic Grammars

12.6. Draw an ATN to implement the grammer of Problem 12.1.

12.7. Given the following parse tree, write down the corresponding context free grammer.



12.8. Create a LISP data structure to model a simple lexicon similar to the one depicted in Figure 12.6.

12.9. Write a LISP match program which checks an input sentence for matching words in the lexicon of the previous problem.

12.10. Derive an ATN for the parse tree of Problem 12.7.

12.11. Derive an ATN (graph) to implement the parse tree of Problem 12.1.

12.12. Determine if the following sentences will be accepted by the grammar of Problem 12.6.
(a) The green green grass of the home
(b) The red car drove in the fast lane.

12.13. Write PROLOG rules to implement the grammar used to derive the parse tree of Problem 12.7. Omit rules for the individual word categories (like noun ([ball | X],X)). Generate a syntax tree using one output parameter.

12.14. Write a PROLOG program that will take grammar rules in the following format:

$$NT \rightarrow (NT \mid T)^*$$

where NT is any nonterminal, T is any terminal, and Kleene star (*) signifies any number of repetitions, and generate the corresponding top-down parser; that is,

```
sentence --> noun_phrase, verb_phrase
determiner --> [the]
```

will generate the following:

```
sentence(I,O) :- noun_phrase(I,R), verb_phrase(R,O).
determiner([the|X],X) :-!.
```

12.15. Modify the program in Problem 12.12 to accept extra arguments used to return meaningful knowledge structures.

```
sentence(sentence(NP,VP)) --> noun_phrase(NP), verb_phrase (VP).
```

12.16. Write a LISP program which uses property lists to create the recursive transition network depicted in Figure 12.9. Each node should be given a name such as S1, N1, and P1 and associated with a list of arc and node pairs emanating from the node.

12.17. Write a recursive program in LISP which tests input sentences for the RTN developed in the previous problem. The program should return *t* if the sentence is acceptable, and nil if not.

12.18. Modify the program of Problem 12.15 to accept sentences of the type depicted in Figure 12.12.

12.19. Write an ATN type of program as depicted in Figure 12.12 which builds structures like those of Figure 12.13.

12.20. Describe in detail the differences between language understanding and language generation. Explain the problems in developing a program which is capable of carrying on a dialog with a group of people.

12.21. Give the processing steps required and corresponding data structures needed for a robot named Rob to formulate instructions for a helper named John to complete a university course add-drop request form.

12.22. Give the conceptual dependency graph for the sentence "Mary drove her car to school" and describe the steps required for a program to transform the sentence to an internal conceptual dependency structure.

13

Pattern Recognition

One of the most basic and essential characteristics of living things is the ability to recognize and identify objects. Certainly all higher animals depend on this ability for their very survival. Without it they would be unable to function even in a static, unchanging environment.

In this chapter we consider the process of computer pattern recognition, a process whereby computer programs are used to recognize various forms of input stimuli such as visual or acoustic (speech) patterns. This material will help to round out the topic of natural language understanding when speech, rather than text, is the language source. It will also serve as an introduction to the following chapter where we take up the general problem of computer vision.

Although some researchers feel that pattern recognition should no longer be considered a part of AI, we believe many topics from pattern recognition are essential to an understanding and appreciation of important concepts related to natural language understanding, computer vision, and machine learning. Consequently, we have included in this chapter a selected number of those topics believed to be important.

13.1 INTRODUCTION

Recognition is the process of establishing a close match between some new stimulus and previously stored stimulus patterns. This process is being performed continually throughout the lives of all living things. In higher animals this ability is manifested in many forms at both the conscious and unconscious levels, for both abstract as well as physical objects. Through visual sensing and recognition, we identify many special objects, such as home, office, school, restaurants, faces of people, handwriting, and printed words. Through aural sensing and recognition, we identify familiar voices, songs and pieces of music, and bird and other animal sounds. Through touch, we identify physical objects such as pens, cups, automobile controls, and food items. And through our other senses we identify foods, fresh air, toxic substances and much else.

At more abstract levels of cognition, we recognize or identify such things as ideas (electromagnetic radiation phenomena, model of the atom, world peace), concepts (beauty, generosity, complexity), procedures (game playing, making a bank deposit), plans, old arguments, metaphors, and so on.

Our pervasive use of and dependence on our ability to recognize patterns has motivated much research toward the discovery of mechanical or artificial methods comparable to those used by intelligent beings. The results of these efforts to date have been impressive, and numerous applications have resulted. Systems have now been developed to reliably perform character and speech recognition; fingerprint and photograph identifications; electroencephelogram (EEG), electrocardiogram (ECG), oil log-well, and other graphical pattern analyses; various types of medical and system diagnoses; resource identification and evaluation (geological, forestry, hydrological, crop disease); and detection of explosive and hostile threats (submarine, aircraft, missile) to name a few.

Object classification is closely related to recognition. The ability to classify or group objects according to some commonly shared features is a form of class recognition. Classification is essential for decision making, learning, and many other cognitive acts. Like recognition, classification depends on the ability to discover common patterns among objects. This ability, in turn, must be acquired through some learning process. Prominent feature patterns which characterize classes of objects must be discovered, generalized, and stored for subsequent recall and comparison.

We do not know exactly how humans learn to identify or classify objects, however, it appears the following processes take place:

New objects are introduced to a human through activation of sensor stimuli. The sensors, depending on their physical properties, are sensitive in varying degrees to certain attributes which serve to characterize the objects, and the sensor output tends to be proportional to the more prominent attributes. Having perceived a new object, a cognitive model is formed from the stimuli patterns and stored in memory. Recurrent experiences in perceiving the same or similar objects strengthen and refine the similarity

patterns. Repeated perception results in the formation of generalized or archetype models of object classes which become useful in matching, and hence recognition, of similar objects.

13.2 THE RECOGNITION AND CLASSIFICATION PROCESS

In artificial or mechanical recognition, essentially the same steps as noted above must be performed. These steps are illustrated in Figure 13.1 and summarized below:

Step 1. Stimuli produced by objects are perceived by sensory devices. The more prominent attributes (such as size, shape, color, and texture) produce the strongest stimuli. The values of these attributes and their relations are used to characterize an object in the form of a pattern vector $\mathbf{X}$, as a string generated by some grammar, as a classification tree, a description graph, or some other means of representation. The range of characteristic attribute values is known as the measurement space M.

Step 2. A subset of attributes whose values provide cohesive object grouping or clustering, consistent with some goals associated with the object classifications, are selected. Attributes selected are those which produce high intraclass and low interclass groupings. This subset represents a reduction in the attribute space dimensionality and hence simplifies the classification process. The range of the subset of attribute values is known as the feature space F.

Step 3. Using the selected attribute values, object or class characterization models are learned by forming generalized phototype descriptions, classification rules, or decision functions. These models are stored for subsequent recognition. The range of the decision function values or classification rules is known as the decision space D.

Step 4. Recognition of familiar objects is achieved through application of the rules learned in Step 3 by comparison and matching of object features with the stored models. Refinements and adjustments can be performed continually thereafter to improve the quality and speed of recognition.

There are two basic approaches to the recognition problem, (1) the decision-theoretic approach and (2) the syntactic approach.

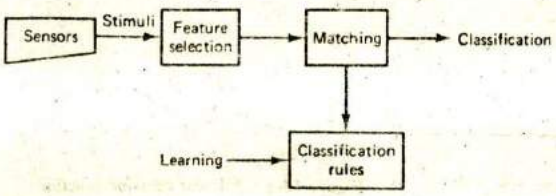


Figure 13.1 The pattern recognition process.

Decision Theoretic Classification

The decision theoretic approach is based on the use of decision functions to classify objects. A decision function maps pattern vectors **X** into decision regions of D. More formally, this problem can be stated as follows.

1. Given a universe of objects $O = \{o_1, o_2, \ldots, o_n\}$, let each o_i have k observable attributes and relations expressable as a vector $\mathbf{V} = (v_1, v_2, \ldots, v_k)$.
2. Determine (a) a subset of $m \leq k$ of the v_i, say $\mathbf{X} = (x_1, x_2, \ldots, x_m)$ whose values uniquely characterize the o_i, and (b) $c \geq 2$ groupings or classifications of the o_i which exhibit high intraclass and low interclass similarities such that a decision function $d(\mathbf{X})$ can be found which partitions D into c disjoint regions. The regions are used to classify each o_i as belonging to at most one of the c classes.

Determining the feature attributes and decision regions requires stipulating or learning mappings from the measurement space M to the feature space F and then a mapping from F to the classification or decision space D,

$$\mathrm{M} \rightarrow \mathrm{F} \rightarrow \mathrm{D}$$

When there are only two classes, say C_1 and C_2, the values of the object's pattern vectors may tend to cluster into two disjoint groups. In this case, a linear decision function $d(\mathbf{X})$ can often be used to determine an object's class. For example, when the classes are clustered as depicted in Figure 13.2, a linear decision function d is adequate to classify unknown objects as belonging to either C_1 or C_2, where

$$d(\mathbf{X}) = w_1 x_1 + w_2 x_2 + w_3$$

The constants w_i in d are parameters or weights that are adjusted to find a separating line for the classes. When a function such as d is used, an object is classified as belonging to C_1 if its pattern vector is such that $d(\mathbf{X}) < 0$, and as

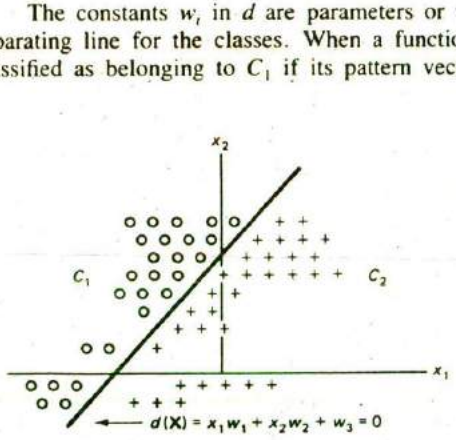


Figure 13.2 A linear decision function.

belonging to class C_2 when $d(\mathbf{X}) > 0$. When $d(\mathbf{X}) = 0$ the classification is indeterminate, so either (or neither) class may be selected.

When class reference vectors, prototypes $\mathbf{R}_j$, $j = 1, \ldots, c$ are available, decision functions can be defined in terms of the distance of the $\mathbf{X}$ from the reference vectors. For example, the distance

$$d_i(\mathbf{X}) = (\mathbf{X} - \mathbf{R}_i)'(\mathbf{X} - \mathbf{R}_i)$$

could be computed for each class C_i, and class C_k would then be chosen when $d_k = \min_i\{C_i\}$.

For the general case of $c \geq 2$ classes, $C_1, C_2, \ldots, C_c$, a decision function may be defined for each class $d_1, d_2, \ldots, d_c$. A class decision rule in this case would be defined to select class C_j when

$$d_j(\mathbf{X}) < d_i(\mathbf{X}) \text{ for } i,j = 1, 2, \ldots, c, \text{ and } i \neq j.$$

When a line d (or more generally a hyperplane in n-space) can be found that separates classes into two or more groups as in the case of Figure 13.2, we say the classes are linearly separable. Classes that overlap each other or surround one another, as in Figure 13.3, cannot generally be classified with the use of simple linear decison functions. For such cases, more general nonlinear (or piecewise-linear) functions may be required. Alternatively, some other selection technique (like heuristics) may be needed.

The decision function approach described above is an example of deterministic recognition since the x_i are deterministic variables. In cases where the attribute values are affected by noise or other random fluctuations, it may be more appropriate to define probabilistic decision functions. In such cases, the attribute vectors $\mathbf{X}$ are treated as random variables, and the decision functions are defined as measures of likelihood of class inclusion. For example, using Bayes' rule, one can compute the conditional probability $P(C_i|\mathbf{X})$ that the class of an object o_j is C_i given the observed value of $\mathbf{X}$ for o_j. This approach requires a knowledge of the prior probability $P(C_i)$, the probability of the occurrence of samples from C_i, as well as $P(\mathbf{X}|C_i)$.

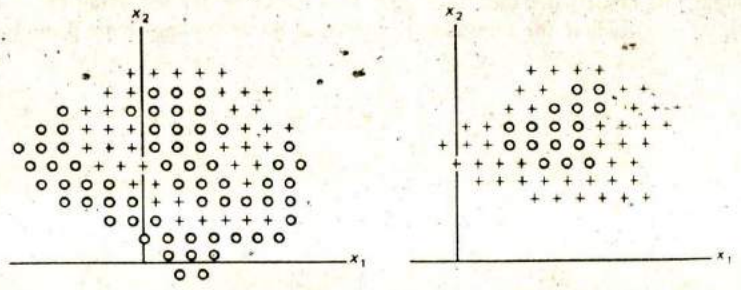


Figure 13.3 Examples of nonlinearly separable classes.

(Note that the C_i are treated like random variables here. This is equivalent to the assumption made in Bayesian classification where the distribution parameter Θ is assumed to be a random variable since C_i may be regarded as a function of Θ). A decision rule for this case is to choose class C_j if

$$P(C_j \mid \mathbf{X}) > P(C_i \mid \mathbf{X}) \text{ for all } i \neq j.$$

A more comprehensive probabilistic approach is one which is based on the use of a loss or risk Bayesian function where the class is chosen on the basis of minimum loss or risk. Let the loss function L_{ij} denote the loss incurred by incorrectly classifying an object actually belonging to class C_i as belonging to C_j. When L_{ij} is a constant for all i, j, $i \neq j$, a decision rule can be formulated using the likelihood ratio defined as (see Chapter 6)

$$\frac{P(\mathbf{X} \mid C_k)}{P(\mathbf{X} \mid C_j)}$$

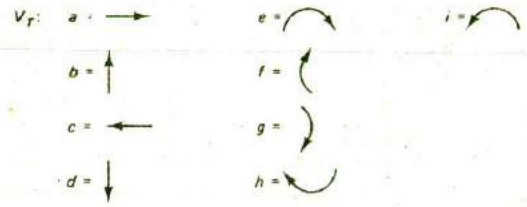
The rule is to choose class C_k whenever the relation

$$\frac{P(\mathbf{X} \mid C_k)}{P(\mathbf{X} \mid C_j)} > \frac{P(C_j)}{P(C_k)} \quad \text{holds for all } j \neq k.$$

Probabilistic decision rules may be constructed as either parametric or nonparametric depending on knowledge of the distribution forms, respectively. For a comprehensive treatment of these methods see (Duda and Hart, 1973) or (Tou and Gonzales, 1974).

Syntactic Classification

The syntactic recognition approach is based on the uniqueness of syntactic "structure" among the object classes. With this approach, a grammar similar to the grammars defined in Chapter 10 or the generative grammars of Chapter 12 is defined for object descriptions. Instead of defining the grammar in terms of an alphabet of characters or terminal words, the vocabulary is based on shape primitives. For example, the objects depicted in Figure 13.4 could be defined using the grammar $G(v_n, v_t, p, s)$, where the terminals v_t consist of the following shape primitives.



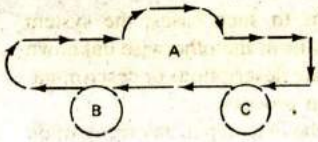


Figure 13.4 Syntactic characterization of objects.

Using syntactic analysis, that is parsing and analyzing the string structures, classification is accomplished by assigning an object to class C_i when the string describing it has been generated by the grammar G_i. This requires that the string be recognized as a member of the language $L(G_i)$. If there are only two classes, it is sufficient to have a single grammar G (two grammars are needed when strings of neither class can occur).

When classification for $c \geq 2$ classes is required, $c - 1$ (or c) different grammars are needed for class recognition. The decision functions in this case are based on grammar recognition functions which choose class C_i if the pattern string is found to be generated by grammar G_i, that is, if it is a member of $L(G_i)$. Patterns not recognized as a member of a defined language are indeterminate.

When patterns are noisy or subject to random fluctuations, ambiguities may occur since patterns belonging to different classes may appear to be the same. In such cases, stochastic or fuzzy grammars may be used. Classification for these cases may be made on the basis of least cost to transform an input string into a valid recognizable string, by the degree of class set inclusion or with a similarity measure using one of the methods described in Chapter 10.

13.3 LEARNING CLASSIFICATION PATTERNS

Before a system can recognize objects, it must possess knowledge of the characteristic features for those objects. This means that the system designer must either build the necessary discriminating rules into the system or the system must learn them. In the case of a linear decision function, the weights that define class boundaries must be predefined or learned. In the case of syntactic recognition, the class grammars must be predefined or learned.

Learning decision functions, grammars, or other rules can be performed in either of two ways, through supervised learning or unsupervised learning. Supervised learning is accomplished by presenting training examples to a learning unit. The examples are labeled beforehand with their correct identities or class. The attribute values and object labels are used by the learning component to inductively extract and determine pattern criteria for each class. This knowledge is used to adjust parameters in decision functions or grammar rewrite rules. Supervised learning concepts are discussed in some detail in Part V. Therefore, we concentrate here on some of the more important notions related to unsupervised learning.

In unsupervised learning, labeled training examples are not available and little

is known beforehand regarding the object population. In such cases, the system must be able to perceive and extract relevant properties from the otherwise unknown objects, find common patterns among them, and formulate descriptions or discrimination criteria consistent with the goals of the recognition process.

This form of learning is known as *clustering*. It is the first step in any recognition process where discriminating features of objects are not known in advance.

Learning through Clustering

Clustering is the process of grouping or classifying objects on the basis of a close association or shared characteristics. The objects can be physical or abstract entities, and the characteristics can be attribute values, relations among the objects, and combinations of both. For example, the objects might be streets, freeways, and other pathways connecting two points in a city, and the classifications, the pathways which provide fast or slow traversal between the points. At a more abstract level, the objects might be some concept such as the quality of the items purchased. The classifications in this case might be made on the basis of some subjective criteria, such as poor, average, or good.

Clustering is essentially a discovery learning process in which similarity patterns are found among a group of objects. Discovery of the patterns is usually influenced by the environment or context and motivated by some goal or objective (even if only for economy in cognition). For example, finding short-cuts between two frequently visited points is motivated by a desire to reduce the planning effort and transit time between the points. Likewise, developing a notion of quality is motivated by a desire to save time and money or to improve one's appearance.

Given different objectives, the same set of objects would, in general, be clustered differently. If the objective given above for the streets, freeways, and the like were modified to include safe for bicycle riding, a different object classification would, in general, result.

Finding the most meaningful cluster groupings among a set of unknown objects o_i requires that similarity patterns be discovered in the feature space. Clustering is usually performed with the intent of capturing any gestalt properties of a group of objects and not just the commonality of certain attribute values. This is one of the basic requirements of *conceptual* clustering (Chapter 19) where the objects are grouped together as members of a concept class. Procedures for conceptual clustering are based on more than simple distance measures. They must also take into account the context (environment) of the objects as well as the goals or objectives of the clustering.

The clustering problem gives rise to several subproblems. In particular, before an implementation is possible, the following questions must be addressed.

1. What set of attributes and relations are most relevant, and what weights should be given to each? In what order should attributes be observed or measured? (If the observation process is sequential, ordering may influence the effectiveness of the attributes in discriminating among objects.)

2. What representation formalism should be used to characterize the objects?

3. What representation scheme should be used to describe the cluster groupings or classifications? Usually, some simplification results if the single representation trick can be used (the use of a single representation method for both object and cluster descriptions).

4. What clustering criteria is most consistent with and effective in achieving the objectives relative to the context or domain? This requires consideration of an appropriate distance or similarity measure compatible with the description domains noted in 2, above.

5. What clustering algorithms can best meet the criteria in 2 within acceptable time and space complexity bounds?

By now questions such as these should be familiar. They are by no means trivial, but they must be addressed when designing a system. They depend on many complex factors for which the tools of earlier chapters become essential. These problems have been addressed elsewhere; therefore, we focus our attention here on the clustering process.

The clustering process must be performed with a limited set of observations, and checking all possible object groupings for patterns is not feasible except with a small number of objects. This is due to the combinatorial explosion which results in arranging n objects into an unknown number m of clusters.[1] Consequently, methods which examine only the more promising groupings must be used. Establishing such groupings requires the use of some measure of similarity, association, or degree of fit among a set of objects.

When the attribute values are real valued, cluster groupings can sometimes be found with the use of point-to-point or point-to-set distances, probability measures (like using the covariance matrix between two populations), scatter matrices, the sum of squared error distance between objects, or other means (see Chapter 10). In these cases, an object is clustered in class C_i if its proximity to other members of C_i is within some threshold or limiting value.

Many clustering algorithms have been proposed for different tasks. One of the most popular algorithms developed at the Stanford Research Institute by G. H. Ball and D. J. Hall (Anderberg, 1973) is known as the ISODATA method. This method requires that the number of clusters m be specified, and threshold values t_1, t_2, and t_3 be given or determined for use in splitting, merging, or discarding

[1] The number of ways in which n objects can be arranged into m groups is an exponential quantity.

$$S_{in} = \left(\frac{1}{m!}\right) \sum_i (-1)^{m-i} \binom{m}{k} k^n.$$

When m is unknown, the number of arrangements increases as the sum of the S_{in}, that is, as S^m. For example when $n = 25$, the number of arrangements is more than $4*10^{18}$.

clusters respectively. During the clustering process, the thresholds are used to determine if a cluster should be split into two clusters, merged with other clusters or discarded (when too small). The algorithm is given with the following steps.

1. Select m samples as seed points for initial cluster centers. This can be done by taking the first m points, selecting random points or by taking the first m points which exceed some mutual minimum separation distance d.
2. Group each sample with its nearest cluster center.
3. After all samples have been grouped, compute new cluster centers for each group. The center can be defined as the centroid (mean value of the attribute vectors) or some similar central measure.
4. If the split threshold t_1 is exceeded for any cluster, split it into two parts and recompute new cluster centers.
5. If the distance between two cluster centers is less than t_2, combine the clusters and recompute new cluster centers.
6. If a cluster has fewer than t_3 members, discard the cluster. It is ignored for the remainder of the process.
7. Repeat steps 3 through 6 until no change occurs among cluster groupings or until some iteration limit has been exceeded.

Measures for determining distances and the center location need not be based on ordered variates. They may be one of the measures described in Chapter 10 (including probabilistic or fuzzy measures) or some measure of similarity between graphs, strings, and even FOPL descriptions. In any case, it is assumed each object o_i is described by a unique point or event in the feature space F.

Up to this point we have ignored the problem of attribute scaling. It is possible that a few large valued variables may completely dominate the other variables in a similarity measure. This could happen, for example, if one variable is measured in units of meters and another variable in millimeters or if the range and scale of variation for two variables are widely different. This problem is closely related to the feature selection problem, that is, in the assignment of weights to feature variables on the basis of their importance or relevance. One simple method for adjusting the scales of such variables is to use a diagonal weight matrix $\mathbf{W}$ to transform the representation vector $\mathbf{X}$ to $\mathbf{X}' = \mathbf{WX}$. Thus, for all of the measures described above, one should assume the representation vectors $\mathbf{X}$ have been appropriately normalized to account for scale variations.

To summarize the above process, a subset of characteristic features which represent the o_i are first selected. The features chosen should be good discriminators in separating objects from different classes, relevant, and measurable (observable) at reasonable cost. Feature variables should be scaled as noted above to prevent any swamping effect when combined due to large valued variables. Next, a suitable metric which measures the degree of association or similarity between objects should be chosen, and an appropriate clustering algorithm selected. Finally, during the

clustering process, the feature variables may need to be weighted to reflect the relative importance of the feature in affecting the clustering.

13.4 RECOGNIZING AND UNDERSTANDING SPEECH

Developing systems that understand speech has been a continuing goal of AI researchers. Speech is one of our most expedient and natural forms of communication, and so understandably, it is a capability we would like AI systems to possess. The ability to communicate directly with programs offers several advantages. It eliminates the need for keyboard entries and speeds up the interchange of information between user and system. With speech as the communication medium, users are also free to perform other tasks concurrently with the computer interchange. And finally, more untrained personnel would be able to use computers in a variety of applications.

The recognition of continuous waveform patterns such as speech begins with sampling and digitizing the waveforms. In this case the feature values are the sampled points $x_i = f(t_i)$ as illustrated in Figure 13.5.

It is known from information theory that a sampling rate of twice the highest speech frequency is needed to capture the information content of the speech waveforms. Thus, sampling requirements will normally be equivalent to 20K to 30K bytes per second. While this rate of information in itself is not too difficult to handle, this, added to the subsequent processing, does place some heavy requirements on real time understanding of speech.

Following sample digitization, the signals are processed at different levels of abstraction. The lowest level deals with phones (the smallest unit of sound), allophones (variations of the phoneme as they actually occur in words), and syllables. Higher level processing deals with words, phrases, and sentences.

The processing approach may be from the bottom, top, or a combination of both. When bottom processing is used the input signal is segmented into basic speech units and a search is made to match prestored patterns against these units. Knowledge about the phonetic composition of words is stored in a lexicon for comparisons. For the top approach, syntax, semantics (the domain), and pragmatics (context) are used to anticipate which words the speaker is likely to have said and

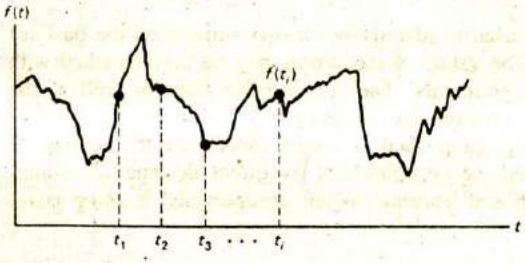


Figure 13.5 Sampling a continuous waveform.

direct the search for recognizable patterns. A combined approach which uses both methods has also been applied successfully.

Early research in speech recognition concentrated on the recognition of isolated words. Patterns of individual words were prestored and then compared to the digitized input patterns. These early systems met with limited success. They were unable to tolerate variations in speaker voices and were highly susceptible to noise. Although important, this early work helped little with the general problem of continuous speech understanding since words appearing as part of a continuous stream differ significantly from isolated words. In continuous speech, words are run together, modified, and truncated to produce a great variation of sounds. Thus, speech analysis must be able to detect different sounds as being part of the same word, but in different contexts. Because of the noise and variability, recognition is best accomplished with some type of fuzzy comparison.

In 1971 the Defense Advanced Research Projects Agency (DARPA) funded a five year program for continuous speech understanding research (SUR). The objective of this research was to design and implement systems that were capable of accepting continuous speech from several cooperative speakers using a limited vocabulary of some 1000 words. The systems were expected to run at slower than real time speeds. A product of this research were several systems including HEARSAY I and II, HARPY, and HWIM. While the systems were only moderately successful in achieving their goals, the research produced other important byproducts as well, particularly in systems architectures, and in the knowledge gained regarding control.

The HEARSAY system was important for its introduction of the blackboard architecture (Chapter 15). This architecture is based on the cooperative efforts of several specialist knowledge components communicating by way of a blackboard in the solution of a class of problems. The specialists are each expert in a different area. For example, speech analysis experts might each deal with a different level of the speech problem. The solution process is opportunistic, with each expert making a contribution when it can. The solution to a given problem is developed as a data structure on the blackboard. As the solution is developed, this data structure is modified by the contributing expert. A description of the systems developed under SUR is given in Barr and Feigenbaum (1981).

13.5 SUMMARY

Pattern recognition systems are used to identify or classify objects on the basis of their attribute and attribute-relation values. Recognition may be accomplished with decision functions or structural grammars. The decision functions as well as the grammars may be deterministic, probabilistic, or fuzzy.

Before recognition can be accomplished, a system must learn the criteria for object recognition. Learning may be accomplished by direct designer encoding, supervised learning, or unsupervised learning. When unsupervised learning is re-

quired, some form of clustering may be performed to learn the object class characteristics.

Speech understanding first requires recognition of basic speech patterns. These patterns are matched against lexicon patterns for recognition. Basic speech units such as phonemes are the building blocks for longer units such as syllables and words.

EXERCISES

13.1. Choose three common objects and determine five of their most discriminating visual attributes.

13.2. For the previous problem, determine three additional nonvisual attributes for the objects which are most discriminating.

13.3. Find a linear decision function which separates the following $x-y$ points into two distinct classes.

−1.8	−5.−1	−3.3	−3.0	1.3	−1.1
0.1	3.4	0.0	2.3	−4.−1	−2.3

13.4. Describe how you would design a pattern recognition program which must validate hand written signatures. Identify some potential problem areas.

13.5. Compare the deterministic decision function approach to the probabilistic decision function apporoach in pattern recognition applications. Give examples when each would be the appropriate function to use.

13.6. Define a set of rewrite rules for a grammar for syntactic generation (recognition) of objects such as the object of Figure 13.4.

13.7. Give two examples of unsupervised learning in humans in which they learn to recognize objects through clustering. Describe how different goals can influence the learning process.

13.8. Apply the ISODATA algorithm to find three different clusters among the following $x-y$ data points.

−2.1	−1.3	−1.2	−1.0	0.2	1.7	1.2	1.1
1.0	2.9	2.8	2.5	2.0	3.9	3.7	3.6
4.8	4.6	4.3	4.2	5.4	5.3	5.2	6.3
6.3	7.7	7.5	7.4	7.2	7.0	−1.6	−2.6

13.9. Consider a cluster algorithm which builds clusters of objects by forming small regions in normalized attribute space (spheres in n-dimensional space) about each object, and includes them in a cluster if and only if the sphere overlaps with at least one other neighboring object sphere. Show how such a scheme could be used to partition the attribute space into subspace with nonlinear boundaries.

13.10. Define an alphabet of shape primitives for a syntactic recognition grammar which

can be used to recognize the integer characters 0, 1, 2, 3, 4, 5, 6, 7, 8, and 9. Check to see that the resultant character strings for each character are unique.

13.11. Give two examples where the single representation trick simplifies clustering among unknown objects.

13.12. Compute the number of ways five objects can be arranged into 1, 2, 3, and 4 groups. From this, try to develop an inductive proof of arranging n objects into m groups.

13.13. Read "High Level Knowledge Sources in Usable Speech Recognition Systems" by Sheryl Young, Alexander Hauptmann, Wayne Ward, Edward Smith, and Philip Werner, in *Communications of the ACM*, Vol. 32, Number 2, Feb., 1989. Summarize some of the more complicated problems associated with general speech recognition.

14

Visual Image Understanding

Vision is perhaps the most remarkable of all of our intelligent sensing capabilities. Through our visual system, we are able to acquire information about our environment without direct contact. Vision permits us to acquire information at a phenomenal rate and at resolutions that are most impressive. For example, one only needs to compare the resolution of a TV camera system to that of a human to see the difference. Roughly speaking, a TV camera has a resolution on the order of 500 parts per square cm, while the human eye has a limiting resolution on the order of some 25×10^6 parts per square cm. Thus, humans have a visual resolution several orders of magnitude better (more than 10,000 times finer) than that of a TV camera. What is even more remarkable is the ease with which we humans sense and perceive a variety of visual images. It is so effortless, we are seldom conscious of the act.

In this chapter, we examine the processes and the problems involved in building computer vision systems. We look at some of the approaches taken thus far and at some of the more successful vision systems constructed to date.

14.1 INTRODUCTION

Because of its wide ranging potential, computer vision has become one of the most intensely studied areas of AI and engineering during the past few decades. Some typical areas of application include the following.